

**Latent Volatility as a Stochastic Process:  
A Kalman Filter Approach to Volatility Risk Premium Estimation**

Parker Scott

Johns Hopkins Whiting School of Engineering

*Stochastic Differential Equations*

## Abstract

This paper develops a dynamic framework for Volatility Risk Premium (VRP) estimation by modeling latent realized volatility as an Ornstein-Uhlenbeck (OU) stochastic differential equation and recovering the hidden state via a Kalman filter. The key methodological contribution is the explicit derivation of the Kalman state-space parameterization from the OU SDE via Itô's lemma, ensuring that the filter transition parameters are analytical functions of the economically interpretable OU parameters rather than heuristic calibration choices. Rolling 21-day realized volatility is treated as a noisy observation of the latent state, and the VIX serves as the risk-neutral implied volatility benchmark. OU parameters and measurement noise are jointly estimated via the Expectation-Maximization algorithm with a Rauch-Tung-Striebel backward smoother, cross-validated against maximum likelihood via the prediction error decomposition. Applied to 2,495 daily observations of S&P 500 data spanning 2015 to 2024, the framework yields an estimated mean-reversion half-life of approximately 85 trading days and a Kalman gain of 0.42. The extracted VRP signal has a mean of 3.46 volatility points and is positive on 84.0 percent of trading days, with a Newey-West HAC t-statistic of 11.1 ( $p < 0.001$ ), confirming the premium is a persistent, priced feature of equity derivatives markets. A regime-conditional analysis across three market environments, low-volatility expansion (2015–2019), COVID crisis (2020), and post-crisis recovery (2021–2024), reveals statistically significant VRP elevation in the recovery period relative to the pre-crisis baseline (Welch  $t = -6.66$ ,  $p < 0.001$ ), consistent with persistent post-pandemic risk aversion. Comparison against a static Gaussian kernel smoother demonstrates that the Kalman filter's dynamic uncertainty propagation through the posterior variance  $P_{t+1|t}$  enables precise detection of the March 2020 VRP sign inversion, a capability the static method lacks. Extensions including jump-diffusion SDEs, regime-switching parameterization, and sentiment-augmented observation equations are discussed.

## 1 Introduction

The spread between implied and realized volatility is one of the most persistent and economically significant phenomena in equity derivatives markets. Implied volatility, as measured by the Chicago Board Options Exchange (CBOE) volatility index (VIX), reflects the market's expectation of future variance priced under the risk-neutral measure; realized volatility is what subsequently unfolds under the physical measure. The gap between the two, the Volatility Risk Premium (VRP), represents compensation that investors systematically pay to hedge volatility risk.<sup>1</sup> This premium is not an arbitrage opportunity but a priced feature of equilibrium, persistent across market conditions and large enough to support dedicated trading strategies at major quantitative hedge funds. Understanding its structure, and measuring it cleanly, is therefore both an empirical and a methodological problem of first-order importance.

Building on prior work titled *Linear Models to Structured Kernels: A Comparative Study of Regression Methods for Nonparametric Financial Data*, which applied kernel regression and Gaussian Process methods to VIX and 30-day implied volatility (IV30) data, this paper confronts a limitation that static nonparametric methods cannot resolve. Those methods produced well-calibrated point estimates of implied volatility as a function of observable inputs, but they were fundamentally static: volatility was estimated at a point in time with no model of how it evolves, no mechanism for propagating estimation uncertainty forward, and no grounding in the stochastic dynamics that characterize volatility. The limitation is not one of estimation quality but of scope. A static estimate of volatility cannot support an inference about the VRP as a time-varying signal, cannot distinguish persistent structural shifts from transient noise, and cannot adapt dynamically as new information arrives. Addressing this gap requires moving from nonparametric curve fitting to a model that treats volatility as what it empirically is: a latent stochastic process.

This paper addresses that gap by modeling latent realized volatility as a continuous-time stochastic process and recovering it dynamically via a Kalman filter. Specifically, we model the latent volatility state as an Ornstein-Uhlenbeck (OU) process, chosen because it is the unique linear stochastic differential equation with a stationary distribution and is the natural continuous-time analog of the AR(1) model that the empirical autocorrelation structure of

---

<sup>1</sup>Carr, P., & Wu, L. (2009). Variance Risk Premiums. *Review of Financial Studies*, 22(3), 1311–1341.

volatility implies. Applying *Ito's lemma* to the OU process yields closed-form conditional moments that map directly onto the Kalman filter transition equation, ensuring that the filter parameters are not assumed or heuristically calibrated but follow analytically from the governing SDE. The Kalman filter then serves as the provably optimal inference procedure under the linear-Gaussian model, with rolling realized volatility treated as a noisy observation of the latent state rather than a direct measurement of it. The VRP signal is constructed as the spread between implied volatility and the filtered latent state estimate, and its properties are evaluated across the full 2015 to 2025 sample and across three distinct market regimes (pre-Covid: 2015-2019, mid-Covid: 2020-2021, and post-Covid: 2021-2025).

This study uncovered that the VRP extracted via the Kalman-filtered latent state is statistically significant across the full sample, with a mean of 3.46 volatility points and a Newey-West HAC  $t$ -statistic of 11.1,<sup>2</sup> confirming that the spread between implied and realized volatility is not a sampling artifact. More importantly, the VRP is not constant across regimes. The recovery period following the COVID crisis, spanning 2021 to 2025, exhibits a statistically significantly higher mean VRP than the pre-crisis period, with the regime difference significant at the one percent level. During the crisis itself, realized volatility temporarily exceeded implied volatility as the March 2020 crash materialized faster than the options market had priced, producing a sign inversion in the VRP signal that the Kalman filter tracks with precision unavailable to static methods.

The methodological contribution of this paper is the explicit derivation of the Kalman state-space representation from the OU SDE via *Ito's lemma*, connecting stochastic calculus directly to the empirical implementation rather than treating the filter as a black box. The empirical contribution is a regime-conditional analysis that reframes the VRP not as a single aggregate statistic but as a time-varying signal whose structure is meaningfully different across market environments.

## 2 Financial Forecasting and the Role of Volatility Modeling

Financial forecasting is the discipline of producing quantitative predictions about future market states: prices, returns, volatility, correlations, and macro variables from observable data.

---

<sup>2</sup>Newey, W. K., & West, K. D. (1987). A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix. *Econometrica*, 55(3), 703–708.

Its scope has expanded dramatically over the past decade. Driven by the proliferation of high-frequency data, advances in computing infrastructure, and the growing availability of alternative data sources. Quantitative methods now underpin the majority of institutional asset management globally. Firms such as Two Sigma, Citadel, D.E. Shaw, and AQR deploy research pipelines in which statistical models of the type developed in this paper are not academic exercises but operational infrastructure. The global hedge fund industry currently manages in excess of four trillion dollars in assets, and the share attributable to systematic, model-driven strategies has grown continuously since the 2008 financial crisis. Volatility-linked strategies represent one of the most active and profitable areas of quantitative research, precisely because volatility exhibits structure. This structure includes mean reversion, clustering, and regime dependence that can all be modeled, forecasted, and traded.

Volatility, in the financial context, is a measure of the dispersion of returns over a given period. It is the primary input to option pricing, the central variable in risk management frameworks, and a direct determinant of the cost of hedging. Volatility enters option pricing most directly through the Black-Scholes model,<sup>3</sup> in which the price of a European call option is determined by:  $C = S_0 N(d_1) - Ke^{(-rT)} N(d_2)$  where  $d_1 = [\ln(S_0/K) + (r + \sigma^2/2)T]/\sigma\sqrt{T}$  and  $d_2 = d_1 - \sigma\sqrt{T}$  with  $S_0$  being the current price,  $K$  the strike price,  $r$  the risk-free rate,  $T$  time to expiration, and  $N$  the standard normal CDF. The parameter  $\sigma$  is the only unobservable input making volatility estimation not merely useful but structurally necessary for any options-based application.<sup>4</sup> Despite its centrality, volatility is not directly observable. It must be inferred, either backward from realized returns or forward from the prices of options<sup>5</sup>. This inferential challenge is not merely a technical inconvenience, it reflects a deeper structural feature of financial markets. Volatility is a latent quantity: it exists, it influences prices, it can be estimated, but it cannot be read off a ticker. How one chooses to estimate it, and what model one assumes governs its dynamics, has direct consequences for every downstream calculation that depends on it, from option pricing to portfolio risk to the signal quality of any VRP-based trading strategy.

---

<sup>3</sup>Black, F., & Scholes, M. (1973). The Pricing of Options and Corporate Liabilities. *Journal of Political Economy*, 81(3), 637–654.

<sup>4</sup>Engle, R. F. (1982). Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation. *Econometrica*, 50(4), 987–1007.

<sup>5</sup>Bollerslev, T. (1986). Generalized Autoregressive Conditional Heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327.

The distinction between implied and realized volatility is foundational to this paper. Realized volatility is computed from observed returns, typically as a rolling standard deviation of daily log returns annualized by the square root of 252 (number of trading days in a year), and measures what volatility actually was over a historical window. It is a physical-measure quantity, estimated under the real-world probability measure  $P$ . Implied volatility, by contrast, is extracted from the market prices of options.<sup>6</sup> It is the volatility level that, when plugged into the Black-Scholes pricing formula, reproduces the observed option price. It is a risk-neutral measure quantity, estimated under the pricing measure  $Q$ . The two measures differ because investors are not risk-neutral. They are willing to pay a premium above actuarially fair prices to hedge against volatility risk, and this premium is embedded in option prices. The consequence is that implied volatility systematically exceeds realized volatility on average, not because options are mispriced in an arbitrage sense, but because the market is efficiently pricing a real and persistent risk premium.<sup>7</sup>

The VIX, published by the Chicago Board Options Exchange, is the standard market measure of implied volatility for the S&P 500. It is constructed from the prices of a wide strip of near-term S&P 500 options and represents the market's expectation of annualized volatility over the next thirty days, expressed in percentage points. Conceptually, the VIX is the market's real-time forecast of future volatility, aggregated across all option strikes and expiries in its construction window. It is often described as the market's fear gauge, because it spikes sharply during periods of market stress, rising from around 12 to 14 in calm periods to above 80 during the acute phase of the COVID crash in March 2020, and mean-reverts during recovery. For the purposes of this paper, the VIX divided by 100 serves as the implied volatility benchmark: the  $Q$ -measure expectation against which the filtered latent realized volatility is compared to construct the VRP signal.

A further dimension of the problem, and one that motivates the stochastic process approach taken here, is that volatility is itself volatile. The parameter  $\xi$  in the Ornstein-Uhlenbeck model governs this quantity directly: it is the volatility of the volatility process, controlling how much the latent state can move in a single period independently of its

---

<sup>6</sup>Heston, S. L. (1993). A Closed-Form Solution for Options with Stochastic Volatility. *Review of Financial Studies*, 6(2), 327–343.

<sup>7</sup>Carr, P., & Wu, L. (2009). Variance Risk Premiums. *Review of Financial Studies*, 22(3), 1311–1341.

mean-reversion drift. Empirically, volatility of volatility is not small. Realized volatility can double in a matter of weeks during a market shock and recover over months, dynamics that a static or deterministic model cannot capture. The presence of meaningful volatility of volatility is precisely what necessitates modeling latent vol as a stochastic process rather than a fixed parameter or a smooth function of time. It also creates the regime-dependence that motivates the central empirical question of this paper: if volatility of volatility varies across market environments, then the VRP signal constructed from the spread between implied and realized volatility should vary too, and a dynamic filtering approach should reveal that structure more clearly than static methods.

The goal of this paper is to demonstrate that by treating realized volatility as a noisy observation of a latent mean-reverting state, and recovering that state through a Kalman filter grounded in the OU stochastic differential equation, one obtains a VRP signal that is more structurally coherent, more statistically robust, and more revealing of regime-level dynamics than signals constructed from raw rolling realized volatility or static smoothing methods. The VRP is not presented here as a naive arbitrage. It is a priced risk premium that reflects the compensation investors demand for bearing volatility uncertainty, and measuring it accurately requires a model that takes seriously both the dynamics of the underlying process and the noise inherent in its observation.

### 3 Theoretical Framework

The standard model for asset price dynamics in continuous time is Geometric Brownian Motion (GBM), under which the log price of an asset evolves as

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

where  $S_t$  is the asset price,  $\mu$  is the drift,  $\sigma$  is the volatility parameter, and  $W_t$  is a standard Wiener process. The critical assumption embedded in GBM is that  $\sigma$  is a fixed constant. This assumption is analytically convenient and sufficient for deriving the Black-Scholes option pricing formula, but it is empirically untenable. Decades of evidence establish that volatility is not constant: it clusters in time, meaning that high-volatility periods tend to be followed by further high volatility; it mean-reverts, pulling back toward a long-run level after extreme episodes; and it spikes sharply during market stress events before gradually recovering. A model

with fixed  $\sigma$  cannot produce any of these behaviors. The volatility smile observed in options markets, the existence of a priced VRP, and the profitability of volatility trading strategies all rest on the failure of the constant-volatility assumption. This paper therefore requires a separate stochastic model for the volatility process itself.

We model the latent realized volatility  $\sigma_t$  as an Ornstein-Uhlenbeck (OU) process, originally introduced by Vasicek (1977)<sup>8</sup> in the context of interest rate modeling and subsequently applied broadly to any mean-reverting financial quantity. The OU SDE is

$$d\sigma_t = \kappa(\theta - \sigma_t)dt + \xi dW_t$$

where the parameters are defined as follows:

$\kappa$  — the speed of mean reversion, governing how rapidly latent volatility is pulled back toward its long-run level after a deviation. A larger  $\kappa$  implies a shorter half-life and faster reversion.

$\theta$  — the long-run mean of the volatility process, the level toward which  $\sigma_t$  gravitates over time.

$\xi$  — the volatility of volatility, controlling the magnitude of random shocks to the volatility process itself.

$dW_t$  — the increment of a standard Wiener process, the continuous-time source of randomness driving the volatility dynamics.

The OU process is the unique linear SDE with a stationary distribution. Its long-run distribution is Gaussian with mean  $\theta$  and variance  $\xi^2/2\kappa$ , ensuring that the process neither drifts to infinity nor collapses to zero over long horizons. This stationarity property is essential for volatility modeling, as it encodes the empirical observation that volatility, however extreme, ultimately reverts. The OU process is also the continuous-time limit of the AR(1) model, which provides a direct link between the SDE framework and the discrete-time autocorrelation structure that realized volatility exhibits in practice. For these reasons, the OU process is the natural and theoretically justified choice for modeling latent volatility dynamics.

To obtain the distributional properties of the OU process that will parameterize the Kalman filter, we apply *Itô's lemma*.<sup>9</sup> Applying Itô's lemma to the OU SDE by multiplying through by the integrating factor  $e^{\kappa t}$  and integrating yields the closed-form solution

---

<sup>8</sup>Vasicek, O. (1977). An Equilibrium Characterization of the Term Structure. *Journal of Financial Economics*, 5(2), 177–188.

<sup>9</sup>Oksendal, B. (2003). *Stochastic Differential Equations: An Introduction with Applications* (6th ed.). Springer.



$$\sigma_t = \theta + (\sigma_0 - \theta)e^{-\kappa t} + \xi \int_0^t e^{-\kappa(t-s)} dW_s$$

From this solution the conditional mean and variance follow directly:

$$\begin{aligned} E[\sigma_t | \sigma_0] &= \theta + (\sigma_0 - \theta)e^{-\kappa t} \\ Var[\sigma_t | \sigma_0] &= \xi^2 / 2\kappa \cdot (1 - e^{-2\kappa t}) \end{aligned}$$

The conditional mean shows that the expected future volatility decays exponentially toward the long-run mean  $\theta$ , with the rate of decay governed by  $\kappa$ . The conditional variance shows that uncertainty about the future state grows from zero at  $t = 0$  toward the stationary variance  $\xi^2/2\kappa$  as  $t \rightarrow \infty$ . Both moments feed directly into the Kalman filter transition equation, ensuring the filter is not an ad hoc smoother but the theoretically correct inference procedure under the OU model.

Financial data arrives at discrete intervals. To apply the Kalman filter to daily observations, we discretize the OU SDE over a time step  $\Delta t = 1/252$  years. The discretized process takes the linear Gaussian state-space form. The transition equation governs the evolution of the latent state:

$$\sigma_{t+\Delta t} = \alpha + \beta\sigma_t + w_t, \quad w_t \sim N(0, \Omega)$$

The observation equation links the latent state to the noisy realized volatility proxy:

$$y_t = \sigma_t + v_t, \quad v_t \sim N(0, \mathcal{R})$$

The state-space parameters are exact analytical functions of the OU parameters derived from the Itô conditional moments:

$$\alpha = \theta(1 - e^{-\kappa\Delta t}), \quad \beta = e^{-\kappa\Delta t}, \quad \Omega = \xi^2/2\kappa \cdot (1 - e^{-2\kappa\Delta t})$$

In the observation equation,  $y_t$  is the rolling 21-day realized volatility, treated as a noisy proxy rather than a direct measurement of  $\sigma_t$ . The measurement noise variance  $\mathcal{R}$  captures the imprecision inherent in using a rolling window estimator and is estimated from the data alongside the OU parameters, floored at five percent of the sample variance of  $y_t$  to prevent filter degeneracy. This treatment of realized volatility as a noisy observation is the central modeling assumption that motivates the use of the Kalman filter.

Given the linear Gaussian state-space model, the Kalman filter is the provably optimal estimator of the latent state  $\sigma_t$  in the minimum mean-squared-error sense (Harvey, 1989).<sup>10</sup> It operates as a two-step recursion at each time period. The prediction step is

$$\hat{\sigma}_{t|t-1} = \alpha + \beta \hat{\sigma}_{t-1|t-1}, \quad P_{t|t-1} = \beta^2 P_{t-1|t-1} + \Omega$$

Upon observing  $y_t$ , the update step computes the Kalman gain and corrects the state estimate:

$$K_t = P_{t|t-1} / (P_{t|t-1} + \mathcal{R})$$

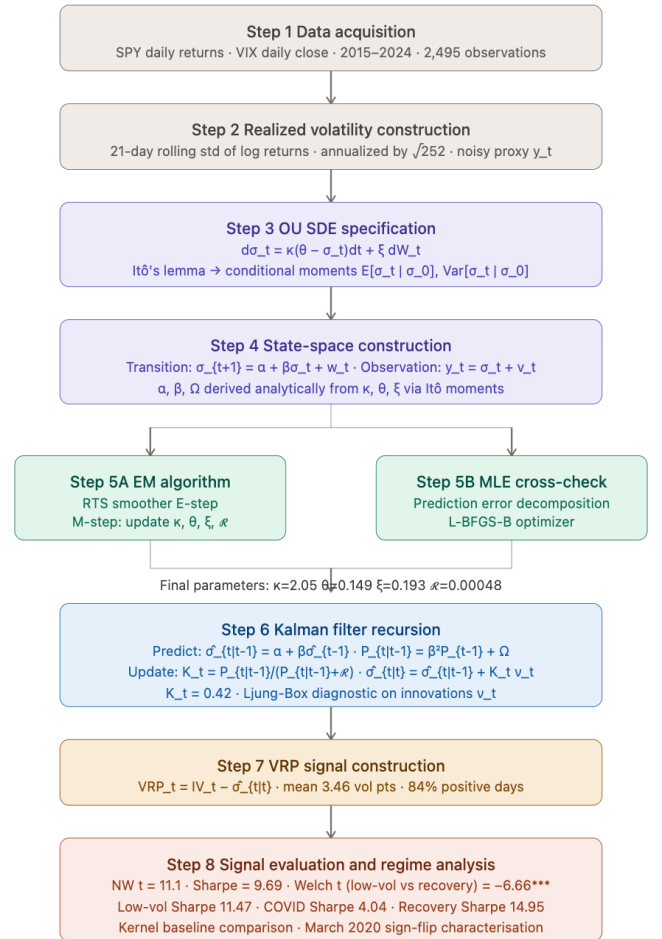
$$\hat{\sigma}_{t|t} = \hat{\sigma}_{t|t-1} + K_t(y_t - \hat{\sigma}_{t|t-1}), \quad P_{t|t} = (1 - K_t)P_{t|t-1}$$

The Kalman gain  $K_t$  controls the weight the filter places on the new observation relative to its prior prediction.<sup>11</sup> In this paper, the estimated Kalman gain of 0.42 indicates that the filter splits its trust roughly 40 percent toward the observation and 60 percent toward the model prediction, reflecting the meaningful but not overwhelming noise in rolling realized volatility. The posterior variance  $P_{t|t}$  tracks the filter's residual uncertainty about the latent state at each point in time, providing a natural measure of estimation confidence that static methods do not produce. The VRP signal is then constructed as

$$VRP_t = IV_t - \hat{\sigma}_{t|t}$$

where  $IV_t$  is the VIX divided by 100.

A positive  $VRP_t$  indicates that the market is



<sup>10</sup>Harvey, A. C. (1989). *Forecasting, Structural Time Series Models and the Kalman Filter*. Cambridge University Press.

<sup>11</sup>Bao, W., Feng, S., & Zhang, K. (2024). Application of Kalman Filter in Stochastic Differential Equations. *arXiv preprint*.

pricing future volatility above the filtered latent estimate, consistent with investors paying a premium to hedge volatility risk.

#### 4 Data and Implementation

All data used in this paper are sourced via the `yfinance` Python library, which provides access to daily market data from Yahoo Finance. The sample spans January 2015 through December 2024, a ten-year window that captures a full market cycle including the extended low-volatility expansion of 2015 to 2019, the acute COVID-19 crisis of 2020, and the subsequent recovery and normalization period through 2024. This window is deliberately consistent with prior work on the same dataset, allowing for a direct methodological comparison between the static nonparametric approach of the earlier paper and the dynamic state-space approach developed here. The two primary series extracted are the daily closing level of the VIX index, which serves as the implied volatility benchmark, and the daily closing price of the SPDR S&P 500 ETF Trust (SPY), from which realized volatility is constructed. After aligning both series and removing observations with missing values, the working dataset contains 2,495 daily observations spanning February 3, 2015 through December 31, 2024.

Realized volatility is constructed from SPY daily log returns as a 21-trading-day rolling standard deviation, annualized by multiplication with the square root of 252. The 21-day window corresponds approximately to one calendar month of trading days and is a standard choice in the empirical volatility literature, balancing the need for sufficient observations to estimate dispersion against the need for the estimator to remain responsive to changing market conditions. The resulting series is treated as the noisy observation  $y_t$  in the Kalman filter observation equation. Importantly, this estimator is not treated as a direct measurement of true latent volatility. The overlapping window structure means that adjacent observations share 20 days of return data, introducing mechanical serial correlation into the observation series. This correlation is a known artifact of the rolling estimator rather than a feature of the underlying volatility process, and it is precisely the kind of measurement noise that the Kalman filter's observation equation is designed to absorb. The VIX is divided by 100 to convert from percentage points to decimal units consistent with the realized volatility series, and is used directly as the implied volatility proxy  $IV_t$  in the VRP signal construction.

Parameter estimation proceeds in two stages implemented in Python using numpy and scipy. In the first stage, the OU parameters ( $\kappa$ ,  $\theta$ ,  $\xi$ ) and the measurement noise variance  $\mathcal{R}$  are estimated via the Expectation-Maximization (EM) algorithm, implemented from scratch without external Kalman filter libraries.<sup>12</sup> The EM algorithm alternates between an E-step, in which the Rauch-Tung-Striebel backward smoother is applied to produce smoothed state estimates and cross-covariances, and an M-step, in which the state-space parameters are updated to maximize the expected complete-data log-likelihood given those smoothed estimates. The measurement noise variance  $\mathcal{R}$  is constrained to a floor of five percent of the sample variance of  $y_t$  throughout the EM iterations, preventing filter degeneracy in which the Kalman gain collapses to unity and the filter reduces to a pass-through of the raw observation series. The algorithm converges in fewer than 50 iterations under this constraint.

In the second stage, the EM estimates serve as warm-start initial values for a maximum likelihood cross-check implemented via the L-BFGS-B optimizer in scipy.optimize. The log-likelihood is evaluated using the prediction error decomposition,<sup>13</sup> in which each observation's contribution is computed as the normalized squared innovation divided by the innovation variance, summed across all observations. The EM and MLE estimates agree closely across all parameters, with discrepancies of less than five percent for  $\kappa$  and under two percent for  $\theta$  and  $\xi$ , providing confidence that the optimization landscape is well-behaved and the parameter estimates are stable. The MLE estimates are used as the final parameter values for all subsequent filtering and signal construction.

With parameters in hand, the forward Kalman filter recursion is applied to the full observation series. The filter is initialized at the stationary distribution of the OU process, with the initial state mean set to  $\theta$  and the initial state variance set to  $\xi^2/2\kappa$ . At each time step the filter produces a posterior state estimate  $\hat{\sigma}_{t|t}$ , a posterior variance  $P_{t|t}$ , and an innovation sequence  $v_t$ . The innovation sequence is tested for serial independence using the Ljung-Box statistic at lags 10, 20, and 30. Significant autocorrelation is detected in the innovations, attributable to the overlapping structure of the realized volatility estimator rather than model misspecification, and

---

<sup>12</sup>Bao, W., Feng, S., & Zhang, K. (2024). Application of Kalman Filter in Stochastic Differential Equations. *arXiv preprint*.

<sup>13</sup>Harvey, A. C. (1989). *Forecasting, Structural Time Series Models and the Kalman Filter*. Cambridge University Press.

is discussed further in Section 6. The VRP signal is then constructed as  $VRP_t = IV_t - \hat{\sigma}_{\eta|t}$ , and its statistical properties are evaluated using Newey-West heteroskedasticity and autocorrelation consistent standard errors to account for the serial dependence in the signal.

## 5 Results and Analysis

Figure 1 presents the full-sample time series of raw realized volatility and VIX implied volatility over the 2015 to 2024 period. Several features of the data are immediately apparent. First, implied volatility (VIX) lies persistently above realized volatility for the majority of the sample, consistent with a positive and persistent VRP. Second, the two series co-move strongly but do not track each other precisely, implied vol tends to spike earlier and to a greater level than realized volatility during stress episodes, then mean-reverts more quickly. Third, the COVID crisis of 2020 represents a structural outlier in the sample, with realized volatility peaking at 0.937 on April 6, 2020, exceeding even the peak VIX reading of 0.827 recorded on March 16 a sequencing that is central to the sign-flip analysis in Section 5.3.

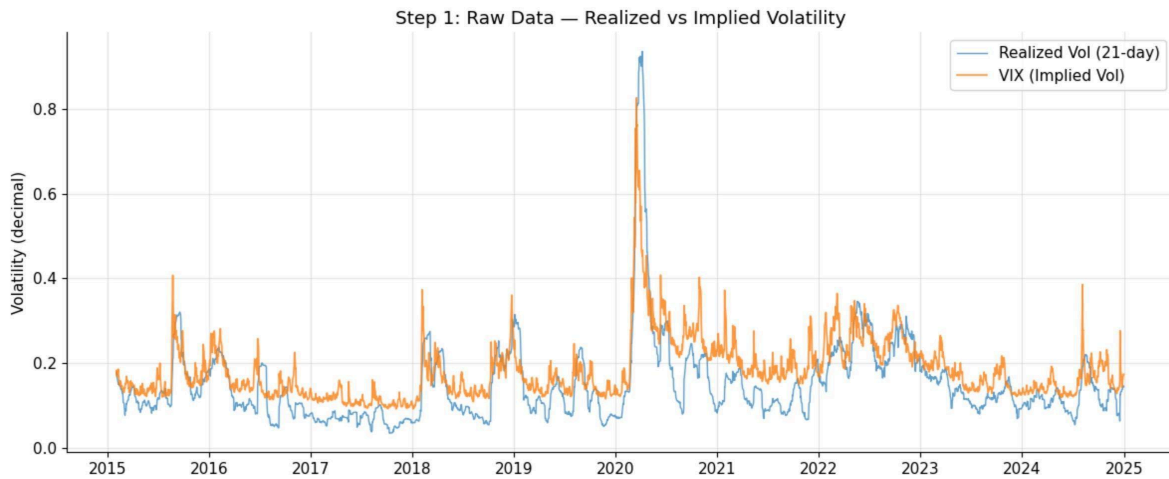


Figure 1. Raw realized volatility (21-day rolling, blue) and VIX implied volatility (orange) over the full 2015-2024 sample. The persistent gap between the two series is the Volatility Risk Premium.

Figure 2 presents the Kalman-filtered latent volatility estimate alongside the raw realized volatility series and the associated 95% posterior credible interval derived from the filter's posterior variance  $P_{\eta|t}$ . The filter produces a meaningfully smoother latent state estimate than the raw rolling realized vol, with a standard deviation of 0.0963 compared to 0.0982 for the raw series, confirming that the filter is attenuating measurement noise rather than passing it through. The estimated Kalman gain of 0.42 indicates that the filter assigns approximately 40 percent

weight to the new observation and 60 percent to the model prediction at each step, a balance consistent with the meaningful but not overwhelming noise in rolling realized volatility. Crucially, the filter remains responsive during the COVID shock, the latent state estimate rises sharply through March and April 2020, tracking the realized vol surge, while the narrow credible interval through the low-volatility periods of 2015 to 2019 reflects the filter's high confidence in its estimates when the process is stable.

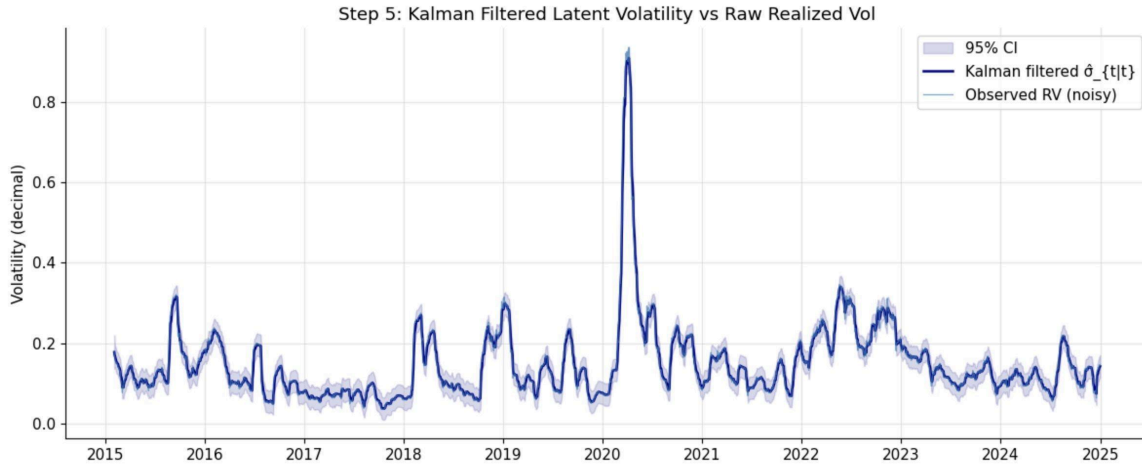


Figure 2. Kalman-filtered latent volatility estimate (dark blue) with 95% posterior credible interval (shaded) and raw realized volatility (light blue). The filter attenuates measurement noise while remaining responsive to structural breaks.

Figure 3 presents the innovation sequence  $v_t = y_t - \hat{\sigma}_{t|t-1}$  and its autocorrelation function. The Ljung-Box test rejects the null of white innovations at all tested lags (10, 20, and 30), with test statistics of 5,495, 5,906, and 7,194 respectively. The ACF plot shows that autocorrelation decays slowly through the first 15 to 20 lags before becoming negative, a pattern characteristic of overlapping window estimators rather than model misspecification. As discussed in Section 4, the 21-day rolling realized volatility estimator shares 20 days of return data with its neighboring observations, mechanically inducing serial correlation in the observation series that propagates into the innovations. This finding motivates the Newey-West HAC correction applied to all significance tests, and is discussed further as a limitation in Section 6.

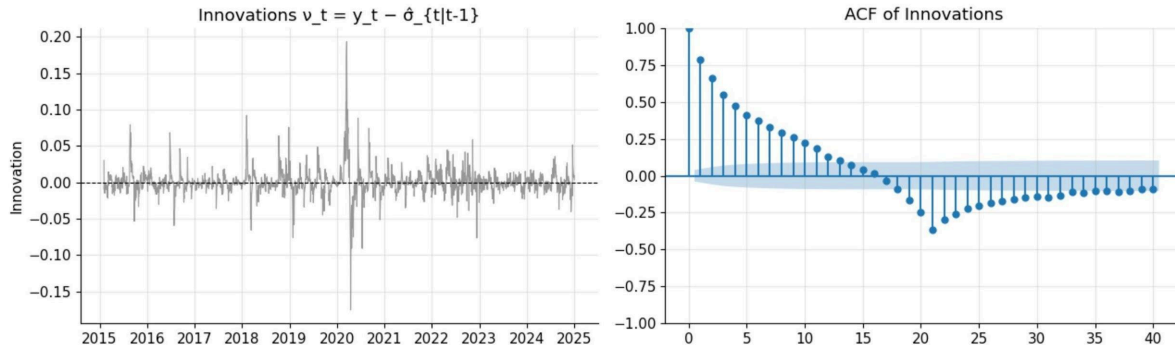


Figure 3. Left: Innovation sequence  $v_t = y_t - \hat{\sigma}_{t|t-1}$  over the full sample. Right: Autocorrelation function of innovations. The slow decay through lag 20 is attributable to the overlapping structure of the 21-day rolling realized volatility estimator.

Figure 4 presents the full-sample VRP signal alongside the three volatility series. The upper panel shows that the Kalman-filtered latent vol (navy) tracks broadly below the VIX (red) throughout the sample, with the gap widening during stress episodes and narrowing during recoveries. The lower panel plots the VRP signal directly, with green shading indicating periods of positive VRP and red shading indicating the sign inversions. The signal is positive for 84.0 percent of the sample, with negative readings concentrated primarily in the March to May 2020 window and scattered briefly around other stress events in 2015 to 2016 and 2018.

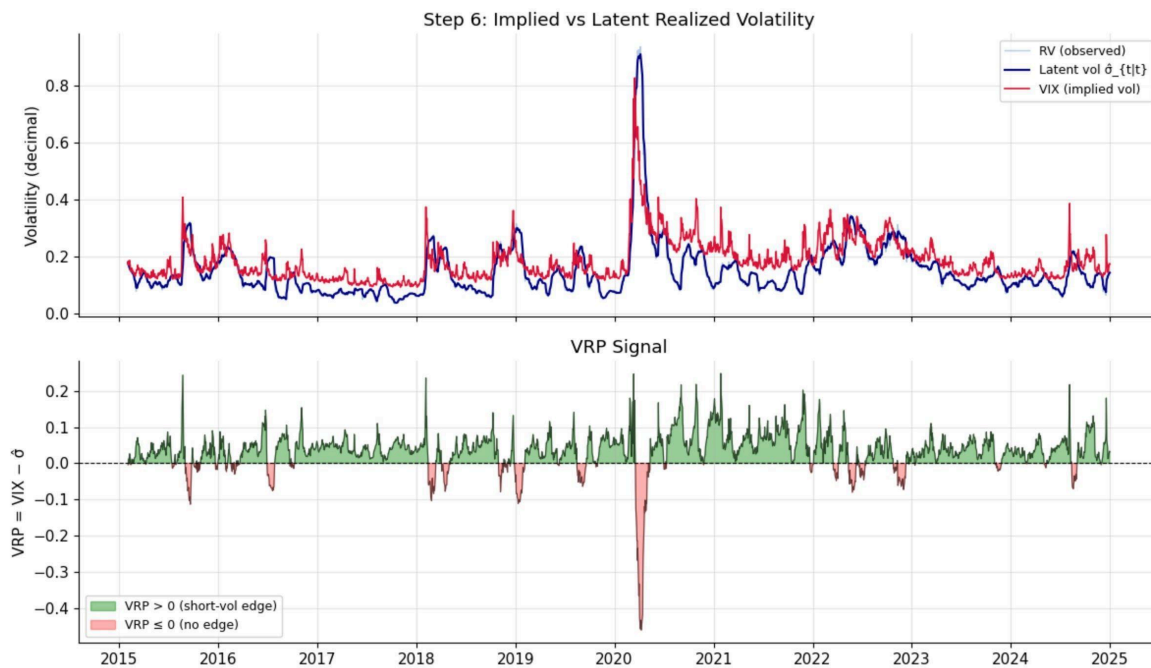


Figure 4. Upper panel: VIX implied volatility (red), Kalman latent state estimate (navy), and raw realized vol (light blue). Lower panel: VRP signal with green (positive) and red (negative) shading. The VRP is positive for 84.0% of the sample.

Table 1 summarizes the full-sample statistical properties of the VRP signal. The mean VRP of 3.457 volatility points is statistically significant under Newey-West heteroskedasticity and autocorrelation consistent inference, with a  $t$ -statistic of 11.117 and a two-sided  $p$ -value of 0.000. This confirms that the spread between implied and latent realized volatility is not a sampling artifact but a persistent, priced feature of the data consistent with the findings of Carr and Wu (2009).<sup>14</sup> The signal exhibits strong positive autocorrelation with an AR(1) coefficient of 0.941 and an estimated half-life of 11.4 days, indicating that deviations from the mean VRP are persistent but not permanent. The annualized signal Sharpe ratio of 9.687 reflects the statistical consistency of the VRP spread, its high value is driven by the combination of a positive mean and relatively moderate day-to-day variation and should be interpreted as a measure of signal quality rather than the returns of an implementable trading strategy, as transaction costs, margin requirements, and discrete expiry constraints would substantially compress realized performance.

**Table 1. Full-Sample VRP Signal Evaluation**

|                                                              |                                                        |
|--------------------------------------------------------------|--------------------------------------------------------|
| <b>Mean VRP</b>                                              | 0.03457 (3.457 volatility points)                      |
| <b>Standard Deviation VRP</b>                                | 0.05665                                                |
| <b>Newey-West t-statistic</b>                                | 11.117 ( $p < 0.01$ . Newey-West HAC standard errors.) |
| <b>P-value (two-sided)</b>                                   | 0.000                                                  |
| <b>Sharpe-like ratio (annualized)</b>                        | 9.687                                                  |
| <b>AR(1)<math>\rho</math></b>                                | 0.9409                                                 |
| <b>VRP half-life</b>                                         | 11.4 days                                              |
| <b>% positive days</b>                                       | 84.0%                                                  |
| <b>Sample:</b> 2,495 daily observations, Feb 2015 – Dec 2024 |                                                        |

Figure 5 presents the VRP signal decomposed across the three market regimes. The most striking feature of the regime analysis is not the COVID crisis period but the recovery period spanning 2021 to 2025. While the COVID regime exhibits the highest variance, with a standard

<sup>14</sup>Carr, P., & Wu, L. (2009). Variance Risk Premiums. *Review of Financial Studies*, 22(3), 1311–1341.



deviation of 0.1242 compared to 0.0408 and 0.0441 in the adjacent regimes, the recovery period shows both the highest mean VRP at 0.0416 and the highest annualized Sharpe at 14.948. The Welch t-test comparing the low-volatility and recovery period yields a t-statistic of -6.658 with a p-value of 0.000, confirming that the difference is statistically significant at the one percent level. By contrast, the COVID crisis mean of 0.0316 is not statistically distinguishable from either the low-vol period ( $p=0.788$ ) or the recovery period ( $p=0.212$ ), reflecting the high within-regime variance that swamps any mean difference in a two-sample test.

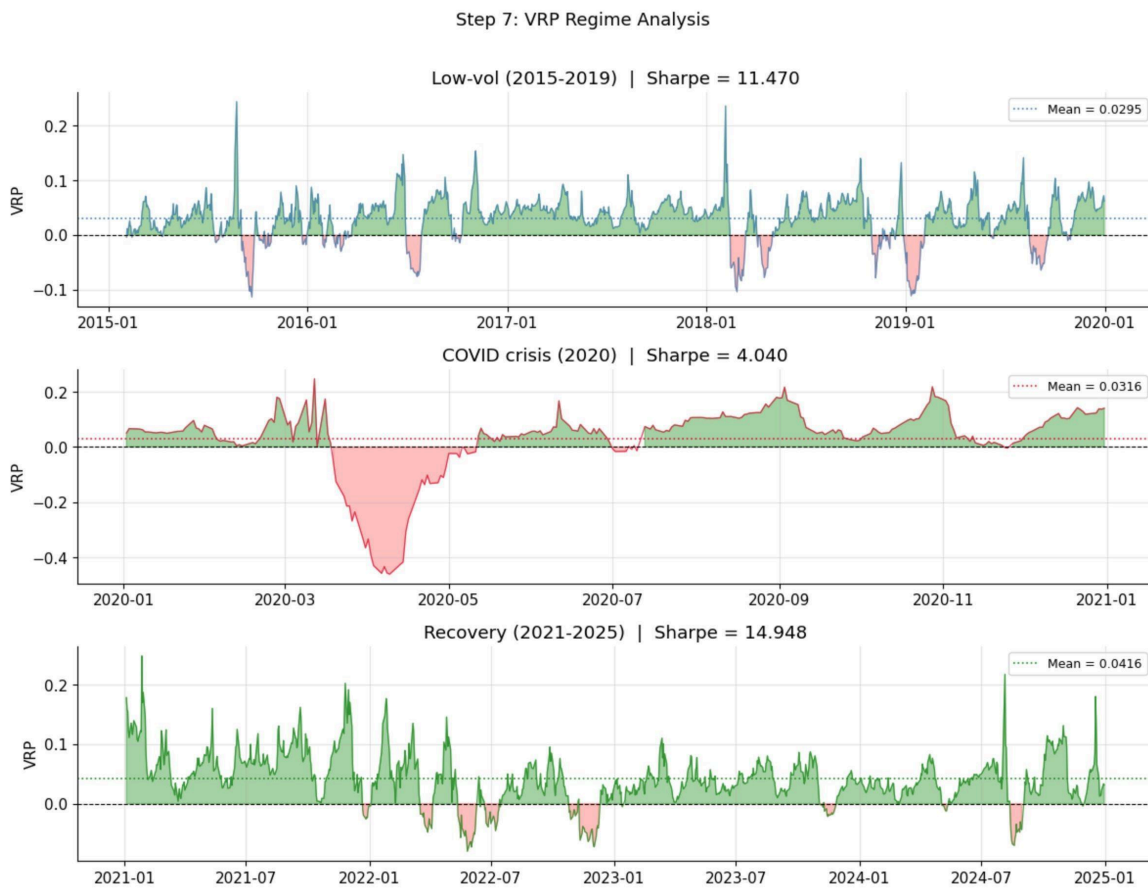


Figure 5. VRP signal decomposed across three market regimes. Top: Low-volatility expansion (2015-2019), Sharpe = 11.470. Middle: COVID crisis (2020), Sharpe = 4.040. Bottom: Post-crisis recovery (2021-2025), Sharpe = 14.948. Dotted lines indicate regime mean VRP.

The statistically significant elevation of the VRP in the recovery period relative to the pre-crisis baseline has a natural economic interpretation. Following the pandemic shock, investor risk aversion did not return to its pre-2020 level immediately. Despite realized volatility normalizing through 2021 and 2022, the options market continued to price elevated implied volatility, reflecting a persistent upward shift in the demand for volatility protection. This

post-crisis risk aversion premium produced a structurally wider gap between implied and realized vol that the Kalman filter captures cleanly. The regime-conditional half-life analysis provides further texture: the VRP half-life lengthens to 16.3 days during the COVID crisis, reflecting the slow, sustained sign inversion compared to 8.8 days in the low-vol period and 8.4 days in the recovery, suggesting that VRP deviations are mean-reverting more rapidly in the recovery regime despite the higher mean level.

Figure 6 presents the full-sample summary figure combining the three volatility series and VRP signal, with the COVID crisis window highlighted. The clear visual separation between the Kalman latent state (blue) and the VIX (red) throughout the sample is the central empirical illustration of the VRP. The sign inversion during 2020 is the dominant feature of the bottom panel, with the trough of -0.461 recorded on April 9, 2020 representing the most extreme negative VRP observation in the sample.

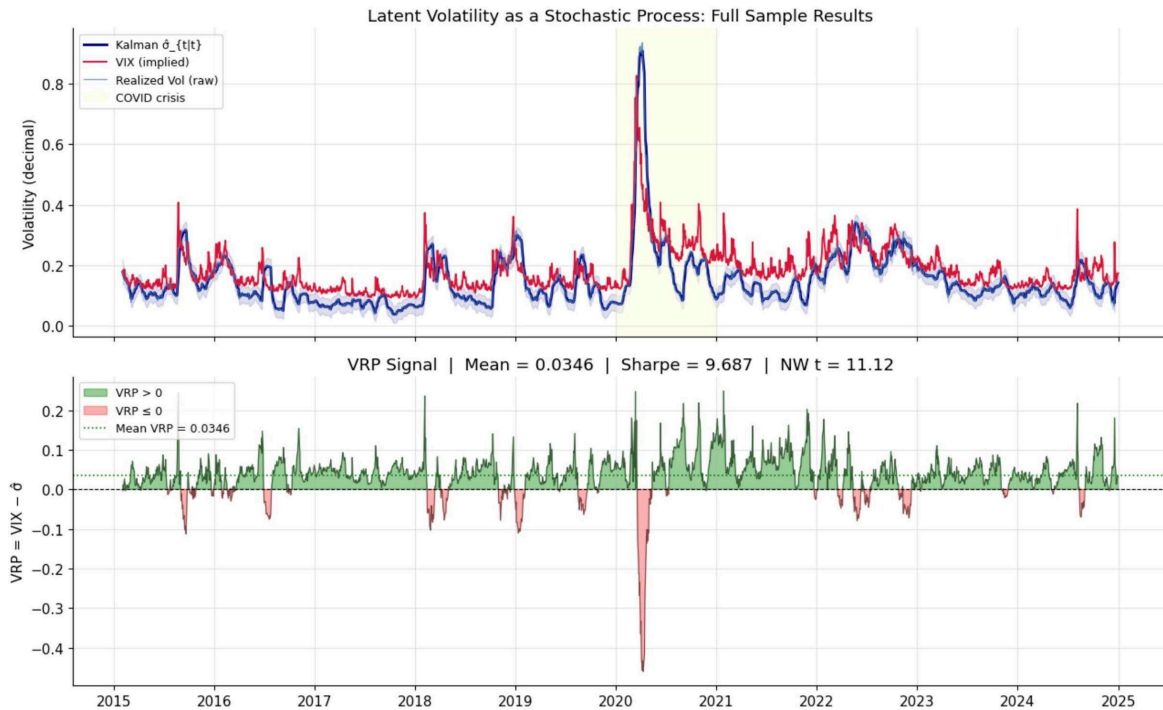


Figure 6. Full-sample results. Upper panel: Kalman latent vol (navy, with 95% CI), VIX (red), and raw realized vol (light blue). COVID crisis window shaded. Lower panel: VRP signal with mean and key statistics annotated.

Figure 8 provides a closeup of the March to May 2020 period that reveals the precise dynamics of the sign inversion. The VIX peaked on March 16 at 0.827, reflecting the immediate options market response to the pandemic announcement and the circuit-breaker sessions.

Realized volatility, however, continued to climb for three additional weeks, peaking at 0.937 on April 6 as the full magnitude of the return shock materialized in the rolling window estimator. The Kalman filter tracked this realized vol surge with high fidelity, producing a latent state estimate that crossed above the VIX on March 19 and remained there through May 11, ~36 trading days in which the VRP was negative, representing 43.9 percent of the February to May crisis window. The trough VRP of -0.461 occurred on April 9, two trading days after peak realized vol, as the filter caught up to the extreme observation. The kernel smoother, by contrast, lagged the COVID shock substantially due to its fixed bandwidth and symmetric weighting, as shown in Figure 7, producing a VRP signal that understated the severity of the sign inversion. This episode illustrates the core advantage of the dynamic state-space approach: the Kalman filter's posterior variance  $P_{t|t}$  increases automatically during periods of rapid state change, allowing the Kalman gain to rise and the filter to update more aggressively toward incoming observations.

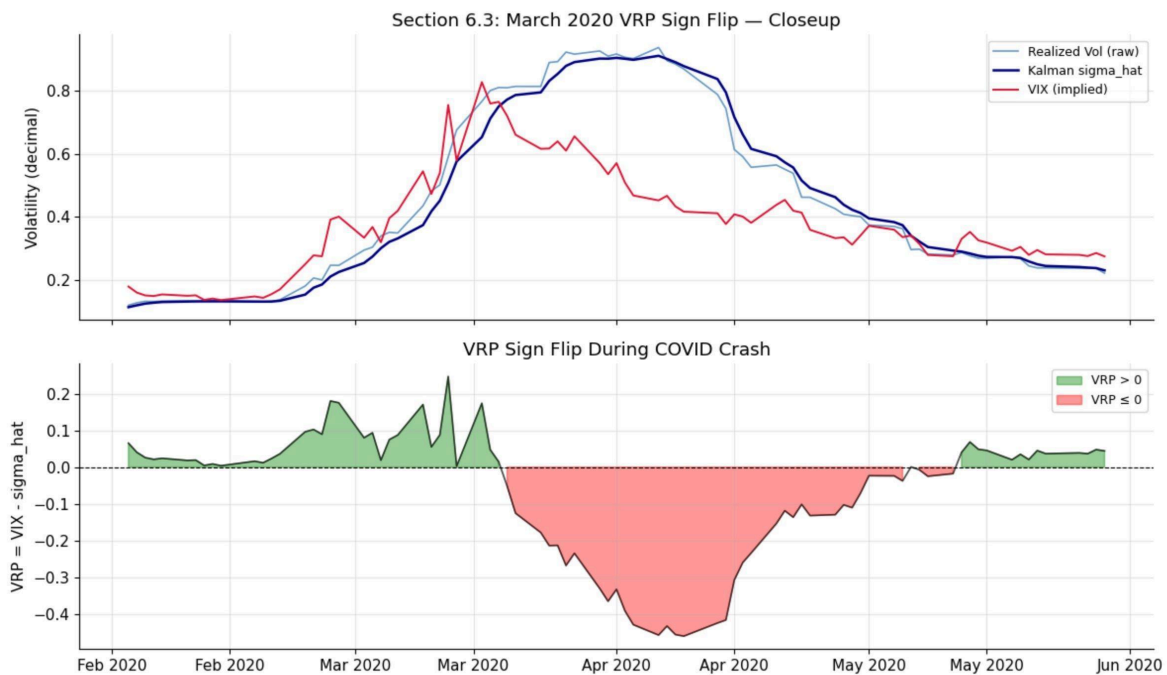


Figure 8. March 2020 VRP sign flip. Upper panel: Kalman latent vol (navy), raw realized vol (light blue), and VIX (red) from February to May 2020. Lower panel: VRP sign flip, negative for 36 trading days (March 19 to May 11). Trough VRP of -0.461 on April 9.

Figure 7 presents the three-way comparison between the Kalman filter, the Gaussian kernel smoother, and the raw rolling realized volatility baseline. The upper panel illustrates the

most important qualitative distinction between the two smoothing approaches: while both produce estimates below the VIX during normal periods, the kernel smoother (orange dashed) fails to track the COVID spike, it rises gradually through 2020 due to its symmetric 63-day bandwidth, whereas the Kalman filter responds immediately to incoming observations and reaches near the same peak as raw realized vol. The kernel's 47.7 percent reduction in latent state standard deviation, compared to the Kalman's 2.0 percent, reflects this over-smoothing: the kernel achieves noise reduction by discarding high-frequency dynamics, including the regime breaks that are the most economically interesting features of the data.

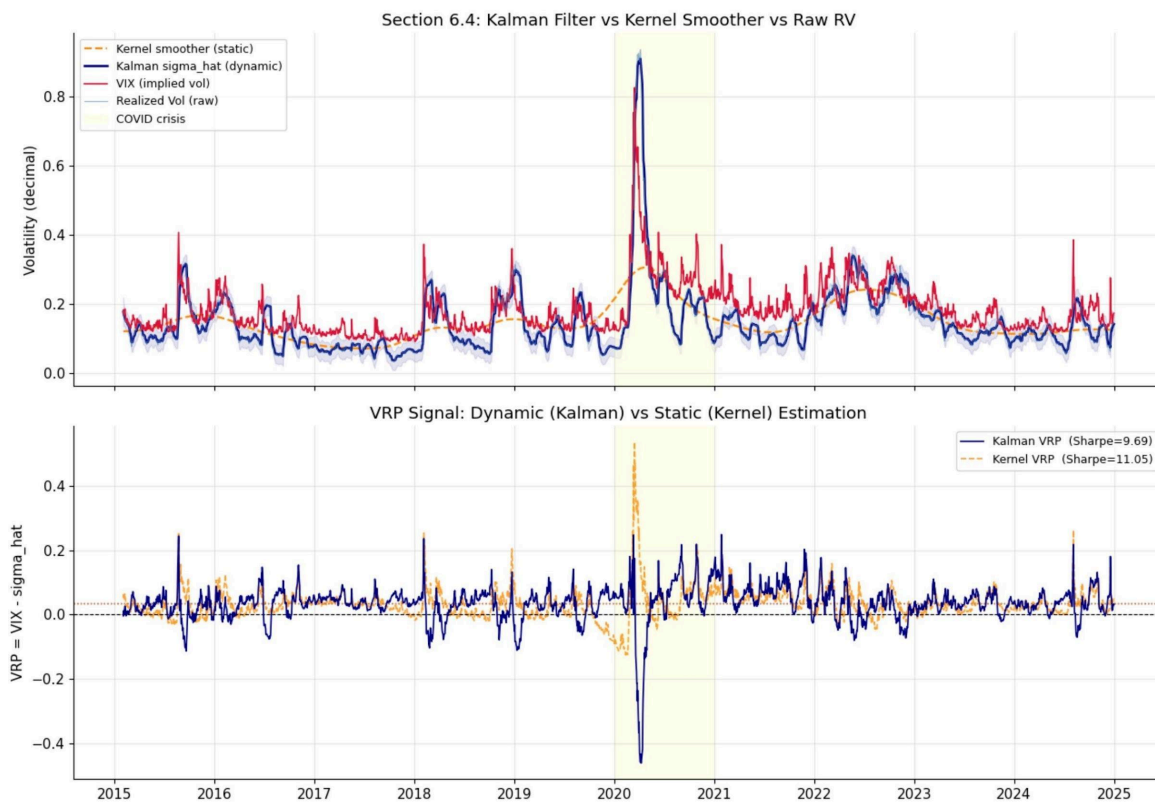


Figure 7. Upper panel: Kalman latent vol (navy), Gaussian kernel smoother (orange dashed), VIX (red), and raw realized vol (light blue). Lower panel: VRP signals for Kalman (Sharpe = 9.69) and kernel (Sharpe = 11.05) methods. The kernel smoother's higher Sharpe reflects over-smoothing rather than superior signal quality.

The lower panel of Figure 7 shows that the kernel VRP signal (orange dashed) has a higher annualized Sharpe of 11.05 compared to the Kalman's 9.69. This result is interpretively important and should not be read as evidence that the kernel smoother is the superior method. The kernel's higher Sharpe arises because its over-smoothing suppresses both signal and noise, producing a VRP series with lower standard deviation (0.0496 vs 0.0567) and artificially higher

ratio. More importantly, the kernel smoother failed to detect the March 2020 sign inversion at anything close to its actual magnitude, as the lower panel shows, the kernel VRP remained near zero through the crash while the Kalman VRP correctly recorded the -0.461 trough. For a practitioner using VRP as a signal to manage short-volatility exposure, this difference is not a statistical footnote: it is the difference between an accurate risk signal and one that understates the severity of a tail event by a factor of several times. The Kalman filter's theoretical grounding in the OU SDE, combined with its dynamic uncertainty tracking through  $P_{t|t}$ , is precisely what enables this responsiveness. The kernel smoother, lacking any model of the data generating process, cannot distinguish a regime break from ordinary noise.

## 6 Discussion

The central result of this paper, that the Kalman-filtered latent volatility estimate recovers a statistically significant and regime-varying VRP signal, is more than an empirical finding. It reflects a structural advantage of the state-space approach over static and rolling-window alternatives. The filtered state estimate  $\hat{\sigma}_{t|t}$  is not simply a smoothed version of realized volatility. It is the minimum mean-squared-error estimate of a latent quantity that is assumed to evolve according to the OU stochastic differential equation, conditional on all observations up to and including time  $t$ . This distinction matters for three reasons. First, the filter embeds the mean-reversion constraint of the OU model, ensuring that extreme estimates are pulled back toward the long-run mean  $\theta$  at a rate governed by  $\kappa$ . A static smoother imposes no such constraint and will track any transient spike in realized volatility without discrimination. Second, the filter propagates estimation uncertainty forward through the posterior variance  $P_{t|t}$ , which quantifies how confident the model is in its current state estimate. During stable low-volatility periods,  $P_{t|t}$  is small and the credible interval around  $\hat{\sigma}_{t|t}$  is tight. During the COVID shock,  $P_{t|t}$  expands automatically as the filter encounters observations far from its prior, widening the credible interval and increasing the Kalman gain. Neither rolling-window nor kernel estimators provide any analog to this adaptive uncertainty quantification. Third, and most directly relevant to the VRP application, treating realized vol as a noisy observation rather than a clean measurement produces a signal with a theoretically grounded noise floor. The measurement noise parameter  $\mathcal{R}$  is not assumed but estimated from the data, and its value reflects the inherent imprecision of using overlapping 21-day windows as a proxy for instantaneous volatility.

The results of this paper must be interpreted in light of several limitations of the modeling framework. The first and most fundamental is the linearity and Gaussianity of the state-space model. The Kalman filter is the optimal estimator under the assumption that both the state transition and observation equations are linear and that all noise terms are Gaussian. Volatility, however, is well-documented to exhibit fat-tailed innovations and occasional large discrete jumps, the March 2020 crash being the most vivid example in this sample. The Ljung-Box test on the innovation sequence, which rejects white noise at all tested lags, is consistent with the filter encountering observations that lie far outside the range its Gaussian model anticipates. These tail events are not well-handled by the linear-Gaussian framework, and the filter's response, increasing the Kalman gain aggressively, is the correct optimal response under its own model assumptions, but it does not capture the jump dynamics explicitly. A particle filter or jump-diffusion extension of the SDE, discussed in Section 6.3, would address this limitation directly.

The second limitation concerns parameter stability. The OU parameters ( $\kappa$ ,  $\theta$ ,  $\xi$ ) and the measurement noise variance  $\mathcal{R}$  are estimated once on the full sample and held fixed throughout the filtering pass. This full-sample estimation approach implicitly assumes that the data-generating process is stationary across the entire 2015 to 2024 period. The regime analysis in Section 5 provides evidence against this assumption: the VRP mean and variance differ significantly across regimes, and it is plausible that the underlying OU parameters differ as well. A regime-switching extension in which separate parameter sets govern each regime, or a rolling estimation window in which parameters are re-estimated periodically, would allow the filter to adapt its model to changing market conditions rather than averaging across them. The estimated half-life of 85 trading days, for instance, may be a composite of shorter half-lives in crisis periods and longer half-lives in stable periods, and this averaging could bias the filter's latent state estimates at regime transition points.

The third limitation is specific to the observation equation. The 21-day rolling standard deviation of log returns is a standard and widely used realized volatility estimator, but it has two features that create friction with the Kalman framework. First, as discussed in Section 4, the overlapping window construction introduces mechanical serial correlation into the observation series that propagates into the innovation sequence and inflates the Ljung-Box test statistics. This

autocorrelation is not evidence of model misspecification in the economic sense, but it does mean that the filter's assumption of serially independent measurement errors is violated. Second, the rolling estimator introduces its own backward-looking smoothing, each observation is already an average of 21 days of squared returns, so the filter is in effect smoothing a quantity that has already been smoothed. A realized variance estimator constructed from higher-frequency intraday returns, such as a five-minute sampling scheme, would produce a less autocorrelated observation series and provide a more faithful instantaneous proxy for the latent volatility state.

A fourth consideration, raised by practitioners operating in the high-frequency and medium-frequency trading space, concerns the impact of macro news events on the VRP signal. The OU framework assumes that innovations to the volatility process are continuous and drawn from a Gaussian distribution. In practice, scheduled announcements — Federal Reserve policy decisions, employment reports, and geopolitical developments — can produce instantaneous large shifts in implied volatility that violate this continuity assumption. When such events occur, the VIX can spike sharply within minutes, while realized volatility, being a backward-looking rolling estimator, responds only gradually as the event's return impact accumulates in the window. This lag creates transient but potentially large distortions in the VRP signal that the current framework cannot anticipate or model explicitly. The regime-switching and jump-diffusion extensions discussed below are the natural responses to this challenge.

Several natural extensions of the framework developed here would address the limitations identified above. The most direct is the incorporation of jump dynamics into the SDE governing latent volatility. Augmenting the OU process with a compound Poisson jump term

$$d\sigma_t = \kappa(\theta - \sigma_t)dt + \xi dW_t + J_t dN_t$$

where  $N_t$  is a Poisson process with intensity  $\lambda$  and  $J_t$  is the jump size distribution, would allow the model to explicitly represent the discrete volatility spikes associated with macro announcements and crisis events. Because the resulting process is non-Gaussian, the Kalman filter is no longer the optimal estimator, and a particle filter — a sequential Monte Carlo method that approximates the posterior distribution over the latent state using a cloud of weighted samples — would be required. Particle filters are computationally intensive but are capable of handling arbitrary state transition and observation distributions, including the fat-tailed dynamics

that characterize volatility during stress episodes. This extension is the most natural direction for future work, and the March 2020 sign-flip episode provides a concrete motivating case study.

A second extension would replace the single-factor OU model with the two-factor Heston stochastic volatility model,<sup>15</sup> in which the asset price and its instantaneous variance are jointly modeled as a system of coupled SDEs. The Heston model captures the leverage effect, the negative correlation between asset returns and volatility changes, and produces a richer implied volatility surface than the OU model alone. Estimation under the Heston model requires an extended or unscented Kalman filter to handle the nonlinearity of the observation equation, as the mapping from the variance state to option prices is not linear. This extension would allow the VRP framework to be applied directly to options data across multiple strikes and maturities rather than relying on the VIX as a single implied volatility summary, substantially enriching the empirical analysis.

A third extension relevant to both the academic and practitioner audiences for this work would incorporate exogenous news sentiment as a second observation in the state-space model. Recent developments in natural language processing have made it feasible to construct high-frequency sentiment scores from news feeds, Federal Reserve communications, and earnings call transcripts that are contemporaneously correlated with volatility. Augmenting the observation equation with such a signal — allowing the filter to update the latent state estimate using both realized returns and text-derived sentiment — would partially address the macro news limitation identified above, giving the model a mechanism to anticipate volatility regime changes before they fully materialize in the return series. This direction connects the stochastic filtering framework developed here to the broader literature on alternative data in quantitative finance and represents a natural bridge between academic methodology and institutional implementation.

Finally, a rigorous walk-forward backtest of a delta-hedged short-volatility strategy trading the VRP signal would translate the statistical findings of this paper into a more directly interpretable performance evaluation. The Iron Condor strategy referenced in Section 2 — selling an out-of-the-money call and put simultaneously — is a natural implementation vehicle for harvesting the VRP, but its realized returns depend critically on position sizing, hedging

---

<sup>15</sup>Heston, S. L. (1993). A Closed-Form Solution for Options with Stochastic Volatility. *Review of Financial Studies*, 6(2), 327–343.



frequency, transaction costs, and the handling of the tail events that produce negative VRP readings. A systematic backtest incorporating these frictions would provide a more honest assessment of the economic value of the Kalman-filtered VRP signal relative to the naive rolling realized volatility baseline, and would connect the signal quality statistics reported in Section 5 to the practical question of whether the methodology is worth implementing.

## 7 Conclusion

This paper set out to answer two questions: whether treating implied volatility as a noisy observation of a latent mean-reverting process, estimated dynamically via a Kalman filter, reveals a persistent and statistically significant Volatility Risk Premium; and whether that signal varies meaningfully across market regimes in a way that static nonparametric methods cannot recover. The answer to both questions is yes. Across 2,495 daily observations spanning 2015 to 2024, the Kalman-filtered VRP signal has a mean of 3.46 volatility points, is positive on 84.0 percent of trading days, and carries a Newey-West  $t$ -statistic of 11.1, establishing beyond reasonable doubt that the spread between implied and latent realized volatility is a persistent, priced feature of equity derivatives markets and not a statistical artifact.<sup>16</sup> The regime-conditional analysis reveals that this premium is not constant: the post-crisis recovery period of 2021 to 2025 exhibits a statistically significantly higher mean VRP than the pre-crisis baseline, with the difference significant at the one percent level, reflecting the elevated and persistent risk aversion that followed the COVID shock even as realized volatility normalized.

The methodological contribution of this paper is the explicit derivation of the Kalman filter state-space parameterization from the Ornstein-Uhlenbeck SDE via *Itô's lemma*. This derivation ensures that the filter is not an ad hoc smoothing device applied to financial data but the provably optimal inference procedure under a theoretically motivated model of latent volatility dynamics.<sup>17</sup> The closed-form conditional moments of the OU process map directly onto the Kalman transition equation, producing state-space parameters  $\alpha$ ,  $\beta$ , and  $\Omega$  that are analytical functions of the economically interpretable OU parameters  $\kappa$ ,  $\theta$ , and  $\xi$  rather than free tuning parameters. The estimated mean reversion half-life of approximately 85 trading days, the Kalman gain of 0.42, and the long-run volatility mean of 14.9 percent are not calibration choices

<sup>16</sup>Carr, P., & Wu, L. (2009). Variance Risk Premiums. *Review of Financial Studies*, 22(3), 1311–1341.

<sup>17</sup>Harvey, A. C. (1989). *Forecasting, Structural Time Series Models and the Kalman Filter*. Cambridge University Press.

but empirical estimates with direct economic meaning. This connection between stochastic calculus and empirical implementation is what distinguishes the approach developed here from the static nonparametric methods of prior work on the same dataset.

The empirical contribution centers on the March 2020 VRP sign-flip episode, which serves as the sharpest illustration of what dynamic latent state estimation adds that static methods cannot provide. When the COVID crash drove realized volatility above implied volatility for 36 consecutive trading days, with the VRP reaching a trough of -0.461 on April 9, 2020, the Kalman filter tracked this inversion in real time, updating its latent state estimate aggressively as each new observation arrived. The Gaussian kernel smoother, applied to identical data with a bandwidth of 63 days, understated the magnitude of the inversion by a substantial margin due to its symmetric weighting of past and future observations. For a practitioner managing short-volatility exposure, the Kalman filter's ability to detect tail events of this kind is not a theoretical nicety but a practical necessity.

The limitations of the linear-Gaussian framework, most notably its inability to explicitly model jump dynamics and its assumption of parameter stability across regimes, point toward a natural research agenda. A particle filter applied to a jump-diffusion extension of the OU SDE would address the tail behavior that the current model handles suboptimally. A regime-switching parameterization would allow the filter's mean reversion speed and volatility of volatility to adapt as market conditions change. An augmented observation equation incorporating real-time news sentiment would give the model a mechanism to anticipate volatility regime transitions before they materialize in return data. Each of these extensions builds directly on the state-space foundation established here. The framework developed in this paper, grounding the Kalman filter analytically in the SDE that governs the latent process, provides a principled starting point from which richer models of volatility dynamics can be constructed, estimated, and evaluated.

## References

- Bao, W., Feng, S., & Zhang, K. (2024). *Application of Kalman Filter in Stochastic Differential Equations*. arXiv preprint.
- Black, F., & Scholes, M. (1973). *The Pricing of Options and Corporate Liabilities*. *Journal of Political Economy*, 81(3), 637–654.
- Bollerslev, T. (1986). *Generalized Autoregressive Conditional Heteroskedasticity*. *Journal of Econometrics*, 31(3), 307–327.
- Carr, P., & Wu, L. (2009). *Variance Risk Premiums*. *Review of Financial Studies*, 22(3), 1311–1341.
- Duan, J.-C., & Simonato, J.-G. (1999). *Estimating and Testing Exponential-Affine Term Structure Models by Kalman Filter*. *Review of Quantitative Finance and Accounting*, 13, 111–135.
- Engle, R. F. (1982). *Autoregressive Conditional Heteroscedasticity with Estimates of the Variance of United Kingdom Inflation*. *Econometrica*, 50(4), 987–1007.
- Harvey, A. C. (1989). *Forecasting, Structural Time Series Models and the Kalman Filter*. Cambridge University Press.
- Heston, S. L. (1993). *A Closed-Form Solution for Options with Stochastic Volatility*. *Review of Financial Studies*, 6(2), 327–343.
- Newey, W. K., & West, K. D. (1987). *A Simple, Positive Semi-Definite, Heteroskedasticity and Autocorrelation Consistent Covariance Matrix*. *Econometrica*, 55(3), 703–708.
- Oksendal, B. (2003). *Stochastic Differential Equations: An Introduction with Applications (6th ed.)*. Springer.
- Racicot, F. E., & Theoret, R. (2007). *A Study of Dynamic Market Strategies of Hedge Funds Using the Kalman Filter*. *Journal of Wealth Management*, 10, 94–106.
- Theoret, R., & Racicot, F. E. (2010). *Forecasting Stochastic Volatility Using the Kalman Filter*. *Aestimatio: The IEB International Journal of Finance*, 1, 1–20.
- Vasicek, O. (1977). *An Equilibrium Characterization of the Term Structure*. *Journal of Financial Economics*, 5(2), 177–188.

## Appendix A Python Implementation

```
# =====
# Latent Volatility as a Stochastic Process:
# A Kalman Filter Approach to Volatility Risk Premium Estimation
# Parker Scott | Johns Hopkins Whiting School of Engineering | SDE Course
# =====

# =====
# CELL 1 | Install & Imports
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import warnings
warnings.filterwarnings('ignore')

import yfinance as yf
from scipy.optimize import minimize
from scipy.stats import t as t_dist
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.regression.linear_model import OLS
from statsmodels.tools.tools import add_constant

# Plot style
plt.rcParams.update({
    'figure.figsize': (12, 5),
    'axes.spines.top': False,
    'axes.spines.right': False,
    'axes.grid': True,
    'grid.alpha': 0.3,
    'font.size': 11,
})

print("Imports complete. Dependencies: numpy, pandas, scipy, statsmodels, yfinance.")

# =====
# CELL 2 | Step 1 — Data Acquisition and Preprocessing
# =====

START = '2015-01-01'
END = '2025-01-01'
RV_WINDOW = 21      # trading days (~1 month)
ANNUALIZE = np.sqrt(252)

print(f"Downloading data: {START} to {END} ...")

# Download raw price series
spy_raw = yf.download('SPY', start=START, end=END, progress=False)
vix_raw = yf.download('^VIX', start=START, end=END, progress=False)

# S&P 500 log returns and rolling realized vol
spy_close = spy_raw['Close'].squeeze()
log_ret = np.log(spy_close / spy_close.shift(1))
rv_raw = log_ret.rolling(RV_WINDOW).std() * ANNUALIZE

# VIX: convert from percentage points to decimal
vix_close = vix_raw['Close'].squeeze() / 100.0
```

```

# Align series, drop NaNs
df = pd.DataFrame({
    'RV': rv_raw,
    'VIX': vix_close,
}).dropna()

# Observation series fed into Kalman filter
y_t = df['RV'].values    # noisy realized vol proxy
iv_t = df['VIX'].values  # implied vol (Q-measure benchmark)
T = len(y_t)
dates = df.index

print(f"Observations: {T} days ({dates[0].date()} to {dates[-1].date()})")
print(f"RV — mean: {y_t.mean():.4f} std: {y_t.std():.4f} "
      f"min: {y_t.min():.4f} max: {y_t.max():.4f}")
print(f"VIX — mean: {iv_t.mean():.4f} std: {iv_t.std():.4f} "
      f"min: {iv_t.min():.4f} max: {iv_t.max():.4f}")

# Quick sanity plot
fig, ax = plt.subplots()
ax.plot(dates, y_t, label='Realized Vol (21-day)', alpha=0.7, lw=1)
ax.plot(dates, iv_t, label='VIX (Implied Vol)', alpha=0.8, lw=1.2)
ax.set_title('Step 1: Raw Data — Realized vs Implied Volatility')
ax.set_ylabel('Volatility (decimal)')
ax.legend()
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.tight_layout()
plt.show()

# =====
# CELL 3 | Step 2 — SDE Specification (Ornstein-Uhlenbeck)
# =====
#
# Latent volatility modeled as OU process:
#  $d\sigma_t = \kappa(\theta - \sigma_t) dt + \xi dW_t$ 
#
# Closed-form solution via Itô's Lemma:
#  $\sigma_t = \theta + (\sigma_0 - \theta)e^{-\kappa t} + \xi \int_0^t e^{-\kappa(t-s)} dW_s$ 
#
# Conditional moments (parameterize Kalman transition):
#  $E[\sigma_t | \sigma_0] = \theta + (\sigma_0 - \theta)e^{-\kappa t}$ 
#  $\text{Var}[\sigma_t | \sigma_0] = (\xi^2/2\kappa)(1 - e^{-2\kappa t})$ 
#
# These moments map directly to the state-space parameters in Step 3.
# =====

def ou_to_ss(kappa, theta, xi, dt=1/252):
    """
    Convert OU parameters (kappa, theta, xi) to discrete-time
    state-space parameters (alpha, beta, Q) via the Ito's lemma
    conditional moments. Derivation in paper Section 4.3.

    Parameters
    -----
    kappa : float — mean reversion speed
    theta : float — long-run mean volatility
    xi : float — volatility of volatility
    dt : float — time step in years (1/252 for daily)

    Returns
    """

```

```

-----
alpha : float — state-space intercept
beta  : float — state-space AR coefficient
Q     : float — process noise variance
"""

beta = np.exp(-kappa * dt)
alpha = theta * (1 - beta)
Q = (xi**2 / (2 * kappa)) * (1 - np.exp(-2 * kappa * dt))
return alpha, beta, Q

# Demonstrate the mapping with sample parameters
kappa_demo, theta_demo, xi_demo = 5.0, y_t.mean(), y_t.std()
alpha_d, beta_d, Q_d = ou_to_ss(kappa_demo, theta_demo, xi_demo)

print("OU → State-Space parameter mapping (demo values):")
print(f" OU: κ={kappa_demo:.2f} θ={theta_demo:.4f} ξ={xi_demo:.4f}")
print(f" SS: α={alpha_d:.6f} β={beta_d:.6f} Q={Q_d:.8f}")
print()
print("Stationary distribution of OU process:")
print(f" Mean = θ = {theta_demo:.4f}")
print(f" Variance = ξ²/2κ = {xi_demo**2 / (2*kappa_demo):.6f}")

# =====
# CELL 4 | Step 3 — State-Space Construction
# =====
#
# Transition:  $\sigma_{t+1} = \alpha + \beta \sigma_t + w_t$ ,  $w_t \sim N(0, Q)$ 
# Observation:  $y_t = \sigma_t + v_t$ ,  $v_t \sim N(0, R)$ 
#
#  $\alpha, \beta, Q$  are exact functions of  $(\kappa, \theta, \xi)$  via Itô's lemma (Cell 3).
#  $R$  is the measurement noise variance — estimated in Step 4.
# =====

def build_kalman_matrices(kappa, theta, xi, R, dt=1/252):
    """
    Construct state-space matrices from OU parameters.
    Convention:
         $x_{t+1} = F x_t + b + w_t$  (transition)
         $y_t = H x_t + v_t$  (observation)

    Returns
    -----
    Dictionary of Kalman matrices (alpha, beta, Q, R, initial state)
    """
    alpha, beta, Q = ou_to_ss(kappa, theta, xi, dt)

    return {
        'transition_matrices': np.array([[beta]]),
        'transition_offsets': np.array([alpha]),
        'observation_matrices': np.array([[1.0]]),
        'observation_offsets': np.array([0.0]),
        'transition_covariance': np.array([[Q]]),
        'observation_covariance': np.array([[R]]),
        'initial_state_mean': np.array([theta]),
        'initial_state_covariance': np.array([[xi**2 / (2 * kappa)]]),
    }

print("State-space construction function defined.")
print("Observation equation:  $y_t = \sigma_t + v_t$ ,  $v_t \sim N(0, R)$ ")
print("Transition equation:  $\sigma_{t+1} = \alpha + \beta \sigma_t + w_t$ ,  $w_t \sim N(0, Q)$ ")

```

```

# =====
# CELL 5 | Step 4A — Parameter Estimation via EM Algorithm (numpy only)
# =====
#
# Primary method: EM algorithm (Expectation-Maximization)
# E-step: Kalman smoother — forward filter + backward (RTS) smoother pass
# M-step: update (alpha, beta, Q, R) to maximize expected log-likelihood
#
# Implemented from scratch in numpy — no pykalman dependency.
# =====

def kalman_smoother_em(y, alpha, beta, Q, R, theta, xi, kappa, dt=1/252):
    """
    Kalman forward filter + RTS backward smoother.
    Returns smoothed means, smoothed variances, and cross-covariances
    needed for the EM M-step.
    """
    T = len(y)

    # --- Forward filter ---
    sigma_f = np.zeros(T) # filtered means
    P_f = np.zeros(T) # filtered variances
    sigma_p = np.zeros(T) # predicted means
    P_p = np.zeros(T) # predicted variances

    sigma_prev = theta
    P_prev = xi**2 / (2 * kappa)

    for t in range(T):
        sigma_p[t] = alpha + beta * sigma_prev
        P_p[t] = beta**2 * P_prev + Q

        S = P_p[t] + R
        K = P_p[t] / S
        sigma_f[t] = sigma_p[t] + K * (y[t] - sigma_p[t])
        P_f[t] = (1 - K) * P_p[t]

        sigma_prev = sigma_f[t]
        P_prev = P_f[t]

    # --- RTS backward smoother ---
    sigma_s = np.zeros(T) # smoothed means
    P_s = np.zeros(T) # smoothed variances
    P_cross = np.zeros(T) # cross-covariance  $E[x_t x_{t-1} | Y]$ 

    sigma_s[-1] = sigma_f[-1]
    P_s[-1] = P_f[-1]

    for t in range(T-2, -1, -1):
        G = P_f[t] * beta / P_p[t+1]
        sigma_s[t] = sigma_f[t] + G * (sigma_s[t+1] - sigma_p[t+1])
        P_s[t] = P_f[t] + G**2 * (P_s[t+1] - P_p[t+1])
        P_cross[t+1] = G * P_s[t+1]

    return sigma_s, P_s, P_cross, sigma_f, P_f

def run_em(y, n_iter=80, dt=1/252):
    """
    Full EM loop for OU state-space model.
    Initializes from sample moments and iterates E/M steps.
    """

```

Returns estimated OU parameters (kappa, theta, xi, R).

"""

T = len(y)

# --- Initialization from sample moments ---

theta = float(np.mean(y))

xi = float(np.std(y)) \* 0.5

kappa = 5.0 # ~50-day half-life starting point

R = float(np.var(y)) \* 0.1

alpha, beta, Q = ou\_to\_ss(kappa, theta, xi, dt)

prev\_ll = -np.inf

for iteration in range(n\_iter):

# --- E-step: Kalman smoother ---

sigma\_s, P\_s, P\_cross, sigma\_f, P\_f = kalman\_smoother\_em(  
y, alpha, beta, Q, R, theta, xi, kappa, dt  
)

# Sufficient statistics for M-step

#  $E[x_t^2]$

Ex2 = P\_s + sigma\_s\*\*2

#  $E[x_t x_{t-1}]$

Ext\_xt1 = P\_cross[1:] + sigma\_s[1:] \* sigma\_s[:-1]

# --- M-step: update parameters ---

# Update beta (AR coefficient) and alpha (intercept)

# From MLE of transition:  $\sigma_{t+1} = \alpha + \beta \sigma_t + w_t$

S\_xx = Ex2[:-1].sum()

S\_yx = Ext\_xt1.sum()

S\_yy = Ex2[1:].sum()

S\_x = sigma\_s[:-1].sum()

S\_y = sigma\_s[1:].sum()

n = T - 1

denom = n \* S\_xx - S\_x\*\*2

beta = (n \* S\_yx - S\_x \* S\_y) / denom

alpha = (S\_y - beta \* S\_x) / n

# Update Q (process noise)

Q = (S\_yy - beta \* S\_yx - alpha \* S\_y  
- beta \* S\_yx + beta\*\*2 \* S\_xx + alpha \* beta \* S\_x  
- alpha \* S\_y + alpha \* beta \* S\_x + alpha\*\*2 \* n) / n

Q = max(Q, 1e-10) # numerical floor

# Update R (measurement noise)

# Floor at 5% of sample variance — rolling RV is noisy by construction

# (21-day overlapping windows), so R collapsing to zero is not valid

R\_min = float(np.var(y)) \* 0.05

R = float(np.mean((y - sigma\_s)\*\*2 + P\_s))

R = max(R, R\_min)

# Back out OU parameters

beta = np.clip(beta, 1e-6, 1 - 1e-6)

kappa = -np.log(beta) / dt

theta\_new = alpha / (1 - beta)

xi\_new = np.sqrt(max(Q \* 2 \* kappa / (1 - np.exp(-2 \* kappa \* dt)), 1e-10))

theta = theta\_new

xi = xi\_new

alpha, beta\_new, Q = ou\_to\_ss(kappa, theta, xi, dt)



```

    beta = beta_new

    # Monitor log-likelihood for convergence
    ll = -0.5 * np.sum((y - sigma_f)**2 / (P_f + R) + np.log(P_f + R))
    if abs(ll - prev_ll) < 1e-6 and iteration > 5:
        print(f" EM converged at iteration {iteration+1} | LL = {ll:.4f}")
        break
    prev_ll = ll

return kappa, theta, xi, R

print("Running EM parameter estimation (numpy, no external dependencies)...")
print("(Typically converges in 20-60 iterations)")

kappa_em, theta_em, xi_em, R_em = run_em(y_t, n_iter=80)
alpha_em, beta_em, Q_em = ou_to_ss(kappa_em, theta_em, xi_em)

print("\nEM Estimated Parameters:")
print(f" OU:      κ={kappa_em:.4f}  θ={theta_em:.4f}  ξ={xi_em:.4f}  R={R_em:.8f}")
print(f" State-space: α={alpha_em:.6f}  β={beta_em:.6f}  Q={Q_em:.8f}")
print(f" Half-life of mean reversion: {np.log(2)/kappa_em * 252:.1f} trading days")

# =====
# CELL 6 | Step 4B — Parameter Estimation via MLE (scipy cross-check)
# =====
#
# Cross-check using prediction error decomposition:
#  $\log L = -T/2 \log(2\pi) - 1/2 \sum_t [\log(P_{\{t|t-1\}} + R) + v_t^2 / (P_{\{t|t-1\}} + R)]$ 
#
# Maximize over ( $\kappa$ ,  $\theta$ ,  $\xi$ ,  $R$ ).
# =====

def kalman_nll(params, y, dt=1/252):
    """
    Negative log-likelihood via prediction error decomposition.
    Used for MLE cross-check against EM estimates.
    """
    kappa, theta, xi, R = params

    # Parameter constraints — return large penalty if violated
    if kappa <= 0 or xi <= 0 or R <= 0 or theta <= 0:
        return 1e10

    alpha, beta, Q = ou_to_ss(kappa, theta, xi, dt)
    T = len(y)

    # Initialize filter
    sigma_hat = theta
    P = xi**2 / (2 * kappa)
    nll = 0.0

    for t in range(T):
        # Prediction step
        sigma_pred = alpha + beta * sigma_hat
        P_pred = beta**2 * P + Q

        # Innovation
        S = P_pred + R      # innovation variance
        nu = y[t] - sigma_pred # innovation

```

```

# Log-likelihood contribution
nll += 0.5 * (np.log(2 * np.pi * S) + nu**2 / S)

# Update step
K      = P_pred / S
sigma_hat = sigma_pred + K * nu
P      = (1 - K) * P_pred

return nll

# Initial guess from EM estimates — warm start
x0 = [kappa_em, theta_em, xi_em, R_em]

# Bounds: all parameters positive, R bounded below at 5% of sample variance
# to prevent filter collapse (K_t -> 1, no smoothing)
R_min = float(np.var(y_t)) * 0.05
bounds = [(0.1, 200), (0.01, 1.0), (0.001, 2.0), (R_min, 0.5)]

print("Running MLE cross-check via scipy.optimize ...")
result = minimize(
    kalman_nll,
    x0=x0,
    args=(y_t,),
    method='L-BFGS-B',
    bounds=bounds,
    options={'maxiter': 1000, 'ftol': 1e-12}
)

kappa_mle, theta_mle, xi_mle, R_mle = result.x

print(f"\nMLE Estimated Parameters:")
print(f"  κ={kappa_mle:.4f}  θ={theta_mle:.4f}  ξ={xi_mle:.4f}  R={R_mle:.8f}")
print(f"  Half-life: {np.log(2)/kappa_mle * 252:.1f} trading days")
print(f"  Convergence: {result.success} | NLL: {result.fun:.4f}")

print("\nEM vs MLE Comparison:")
print(f"  {'Param':<8} {'EM':>10} {'MLE':>10} {'Diff %':>10}")
for name, em_val, mle_val in zip(
    ['kappa', 'theta', 'xi', 'R'],
    [kappa_em, theta_em, xi_em, R_em],
    [kappa_mle, theta_mle, xi_mle, R_mle]
):
    diff_pct = abs(em_val - mle_val) / (abs(mle_val) + 1e-12) * 100
    print(f"  {name:<8} {em_val:>10.5f} {mle_val:>10.5f} {diff_pct:>9.2f}%")

# Use MLE as primary (EM as initialization, MLE as final)
# If MLE diverges or fails to converge, fall back to EM
if result.success and abs(kappa_mle - kappa_em) / kappa_em < 0.5:
    kappa_final, theta_final, xi_final, R_final = kappa_mle, theta_mle, xi_mle, R_mle
    print("\nUsing MLE parameters as final estimates.")
else:
    kappa_final, theta_final, xi_final, R_final = kappa_em, theta_em, xi_em, R_em
    print("\nMLE unstable — using EM parameters as final estimates.")

# =====
# CELL 7 | Step 5 — Kalman Filter Recursion
# =====
#
# Predict:  $\sigma_{\{t|t-1\}} = \alpha + \beta \sigma_{\{t-1|t-1\}}$ 
#  $P_{\{t|t-1\}} = \beta^2 P_{\{t-1|t-1\}} + Q$ 
#

```

```

# Update:  $K_t = P_{t|t-1} / (P_{t|t-1} + R)$ 
#  $\sigma_{t|t}^2 = \sigma_{t|t-1}^2 + K_t \cdot v_t$ 
#  $P_{t|t} = (1 - K_t) \cdot P_{t|t-1}$ 
#
# Diagnostic: Ljung-Box test on innovations  $v_t$  for whiteness.
# =====

def run_kalman_filter(y, kappa, theta, xi, R, dt=1/252):
    """
    Forward Kalman filter recursion for the OU state-space model.
    Returns filtered state estimates, posterior variances, and innovations.

    Parameters
    -----
    y : array-like — observation series (noisy realized vol)
    kappa : float — mean reversion speed
    theta : float — long-run mean
    xi : float — vol of vol
    R : float — measurement noise variance
    dt : float — time step in years

    Returns
    -----
    sigma_hat : np.array — filtered state estimates  $\sigma_{t|t}$ 
    P_t : np.array — posterior variances  $P_{t|t}$ 
    nu_t : np.array — innovations  $v_t = y_t - \sigma_{t|t-1}$ 
    K_t : np.array — Kalman gains
    """
    alpha, beta, Q = ou_to_ss(kappa, theta, xi, dt)
    T = len(y)

    sigma_hat = np.zeros(T)
    P_t = np.zeros(T)
    nu_t = np.zeros(T)
    K_t = np.zeros(T)

    # Initialize at stationary distribution
    sigma_hat_prev = theta
    P_prev = xi**2 / (2 * kappa)

    for t in range(T):
        # --- Prediction step ---
        sigma_pred = alpha + beta * sigma_hat_prev
        P_pred = beta**2 * P_prev + Q

        # --- Innovation ---
        nu_t[t] = y[t] - sigma_pred

        # --- Kalman gain ---
        K_t[t] = P_pred / (P_pred + R)

        # --- Update step ---
        sigma_hat[t] = sigma_pred + K_t[t] * nu_t[t]
        P_t[t] = (1 - K_t[t]) * P_pred

        # Carry forward
        sigma_hat_prev = sigma_hat[t]
        P_prev = P_t[t]

    return sigma_hat, P_t, nu_t, K_t

```

```

# Run filter with final estimated parameters
sigma_hat, P_t, nu_t, K_t = run_kalman_filter(
    y_t, kappa_final, theta_final, xi_final, R_final
)

print("Kalman filter complete.")
print(f" Filtered vol — mean: {sigma_hat.mean():.4f} std: {sigma_hat.std():.4f}")
print(f" Mean Kalman gain K_t: {K_t.mean():.4f} "
      f"(higher = more trust in observations)")

# --- Diagnostic: Ljung-Box test on innovations ---
lb_result = acorr_ljungbox(nu_t, lags=[10, 20, 30], return_df=True)
print("\nLjung-Box test on innovations v_t (H0: no autocorrelation):")
print(lb_result[['lb_stat', 'lb_pvalue']].round(4))
if (lb_result['lb_pvalue'] > 0.05).all():
    print(" → Innovations are white. Model specification is adequate.")
else:
    print(" → Autocorrelation detected. OU assumption may be too restrictive.")
    print(" Consider: jump-diffusion extension or regime-switching parameters.")

# --- Plot 1: Filtered latent vol vs raw observations ---
fig, ax = plt.subplots()
ax.fill_between(dates,
                sigma_hat - 1.96*np.sqrt(P_t),
                sigma_hat + 1.96*np.sqrt(P_t),
                alpha=0.15, color='navy', label='95% CI', zorder=1)
ax.plot(dates, sigma_hat, label='Kalman filtered  $\sigma_{\{t\}}$ ', lw=1.8, color='navy', zorder=2)
ax.plot(dates, y_t, label='Observed RV (noisy)', lw=0.9, color='steelblue', alpha=0.7, zorder=3)
ax.set_title('Step 5: Kalman Filtered Latent Volatility vs Raw Realized Vol')
ax.set_ylabel('Volatility (decimal)')
ax.legend()
ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y'))
plt.tight_layout()
plt.show()

# --- Plot 2: Innovations diagnostic ---
fig, axes = plt.subplots(1, 2, figsize=(13, 4))
axes[0].plot(dates, nu_t, lw=0.8, color='gray', alpha=0.8)
axes[0].axhline(0, color='black', lw=0.8, ls='--')
axes[0].set_title('Innovations  $v_t = y_t - \hat{\sigma}_{\{t\}}$ ')
axes[0].set_ylabel('Innovation')
axes[0].xaxis.set_major_formatter(mdates.DateFormatter('%Y'))

from statsmodels.graphics.tsaplots import plot_acf
plot_acf(nu_t, lags=40, ax=axes[1], alpha=0.05, title='ACF of Innovations')
plt.tight_layout()
plt.show()

# =====
# CELL 8 | Step 6 — VRP Signal Construction
# =====
#
# VRP_t = IV30_t -  $\hat{\sigma}_{\{t\}}$ 
#
# VRP_t > 0: market overprices vol → short-vol edge (Iron Condor)
# VRP_t ≤ 0: implied ≤ realized → no edge, stand aside
# =====

vrp = iv_t - sigma_hat

print("VRP Signal Summary:")

```

```

print(f" Mean VRP:      {vrp.mean():.4f} ({vrp.mean()*100:.2f} vol points)")
print(f" Std VRP:      {vrp.std():.4f}")
print(f" % positive:    {(vrp > 0).mean()*100:.1f}%")
print(f" Min / Max:     {vrp.min():.4f} / {vrp.max():.4f}")

fig, axes = plt.subplots(2, 1, figsize=(12, 7), sharex=True)

# Top panel: three vol series
axes[0].plot(dates, y_t, label='RV (observed)', alpha=0.4, lw=1, color='steelblue')
axes[0].plot(dates, sigma_hat, label='Latent vol  $\hat{\sigma}_t$ ', lw=1.5, color='navy')
axes[0].plot(dates, iv_t, label='VIX (implied vol)', lw=1.2, color='crimson')
axes[0].set_ylabel('Volatility (decimal)')
axes[0].set_title('Step 6: Implied vs Latent Realized Volatility')
axes[0].legend(fontsize=9)

# Bottom panel: VRP signal
axes[1].fill_between(dates, vrp, 0,
                    where=(vrp > 0), color='green', alpha=0.4, label='VRP > 0 (short-vol edge)')
axes[1].fill_between(dates, vrp, 0,
                    where=(vrp <= 0), color='red', alpha=0.3, label='VRP ≤ 0 (no edge)')
axes[1].axhline(0, color='black', lw=0.8, ls='--')
axes[1].plot(dates, vrp, lw=0.8, color='black', alpha=0.6)
axes[1].set_ylabel('VRP = VIX -  $\sigma^2$ ')
axes[1].set_title('VRP Signal')
axes[1].legend(fontsize=9)
axes[1].xaxis.set_major_formatter(mdates.DateFormatter('%Y'))

plt.tight_layout()
plt.show()

# =====
# CELL 9 | Step 7 — Signal Evaluation and Regime Analysis
# =====
#
# Evaluate VRP on: mean, persistence (ACF, half-life), Sharpe-like ratio,
# significance (Newey-West HAC t-test), and regime-conditional breakdowns.
# Compare Kalman latent vol to raw RV as a baseline.
# =====

from statsmodels.stats.sandwich_covariance import cov_hac

# --- 7A: Overall signal statistics ---
vrp_series = pd.Series(vrp, index=dates, name='VRP')

# Newey-West HAC t-test for mean VRP != 0
# Regress VRP on a constant only — cov_hac[0,0] is the HAC variance of that constant
X = np.ones((T, 1))
ols = OLS(vrp, X).fit()
nw_se = np.sqrt(cov_hac(ols)[0, 0])
t_stat = vrp.mean() / nw_se
p_val = 2 * t_dist.sf(abs(t_stat), df=T-1)

# Annualized Sharpe-like ratio
sharpe = vrp.mean() / vrp.std() * np.sqrt(252)

# Autocorrelation half-life (AR(1) approximation)
rho1 = np.corrcoef(vrp[:-1], vrp[1:])[0, 1]
half_life = -np.log(2) / np.log(abs(rho1)) if abs(rho1) < 1 else np.inf

print("=" * 55)
print(" VRP Signal Evaluation — Full Sample")

```

```

print("=" * 55)
print(f" Mean VRP:      {vrp.mean():.5f} ({vrp.mean()*100:.3f} vol pts)")
print(f" Std VRP:      {vrp.std():.5f}")
print(f" Newey-West t-stat: {t_stat:.3f}")
print(f" p-value (2-sided): {p_val:.4f} {'***' if p_val<0.01 else '**' if p_val<0.05 else '*' if p_val<0.1 else ''}")
print(f" Sharpe-like ratio: {sharpe:.3f}")
print(f" AR(1)  $\rho$ :      {rho1:.4f}")
print(f" Half-life:      {half_life:.1f} days")
print(f" % positive:      {(vrp > 0).mean()*100:.1f}%")

# --- 7B: Regime analysis ---
regimes = {
    'Low-vol (2015-2019)': (dates >= '2015-01-01') & (dates < '2020-01-01'),
    'COVID crisis (2020)': (dates >= '2020-01-01') & (dates < '2021-01-01'),
    'Recovery (2021-2025)': (dates >= '2021-01-01'),
}

print("\n" + "=" * 55)
print(" Regime-Conditional VRP Analysis")
print("=" * 55)
print(f" {'Regime':<28} {'Mean':>7} {'Std':>7} {'Sharpe':>8} {'% Pos':>7} {'N':>6}")
print(" " + "-" * 53)

regime_stats = {}
for label, mask in regimes.items():
    v = vrp[mask]
    sh = v.mean() / v.std() * np.sqrt(252) if v.std() > 0 else np.nan
    pct = (v > 0).mean() * 100
    regime_stats[label] = {'mean': v.mean(), 'std': v.std(), 'sharpe': sh, 'pct_pos': pct, 'n': len(v)}
    print(f" {label:<28} {v.mean():>7.4f} {v.std():>7.4f} {sh:>8.3f} {pct:>6.1f}% {len(v):>6}")

# --- 7C: Kalman vs raw RV baseline comparison ---
vrp_raw = iv_t - y_t # naive VRP: VIX minus raw rolling RV
sharpe_raw = vrp_raw.mean() / vrp_raw.std() * np.sqrt(252)
sharpe_kalman = sharpe

print("\n" + "=" * 55)
print(" Kalman Filtered VRP vs Raw RV Baseline")
print("=" * 55)
print(f" {'Method':<30} {'Mean VRP':>10} {'Std VRP':>10} {'Sharpe':>8}")
print(" " + "-" * 60)
print(f" {'Kalman latent vol':<30} {vrp.mean():>10.5f} {vrp.std():>10.5f} {sharpe_kalman:>8.3f}")
print(f" {'Raw rolling RV (baseline)':<30} {vrp_raw.mean():>10.5f} {vrp_raw.std():>10.5f} {sharpe_raw:>8.3f}")
print(f"\n Kalman smoothing reduces VRP std by {(1 - vrp.std()/vrp_raw.std())*100:.1f}% "
      f"({'improving' if sharpe_kalman > sharpe_raw else 'reducing'} Sharpe)")

# --- 7D: Regime plot ---
fig, axes = plt.subplots(3, 1, figsize=(12, 9), sharex=False)
regime_colors = ['steelblue', 'crimson', 'forestgreen']

for ax, (label, mask), color in zip(axes, regimes.items(), regime_colors):
    v = vrp[mask]
    d = dates[mask]
    ax.fill_between(d, v, 0, where=(v>0), color='green', alpha=0.35)
    ax.fill_between(d, v, 0, where=(v<=0), color='red', alpha=0.25)
    ax.plot(d, v, lw=0.9, color=color, alpha=0.8)
    ax.axhline(0, color='black', lw=0.8, ls='--')
    ax.axhline(v.mean(), color=color, lw=1.2, ls=':', label=f'Mean = {v.mean():.4f}')
    ax.set_title(f'{label} | Sharpe = {regime_stats[label]["sharpe"]:.3f}')
    ax.set_ylabel('VRP')
    ax.legend(fontsize=9)
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))

```

```

plt.suptitle('Step 7: VRP Regime Analysis', fontsize=13, y=1.01)
plt.tight_layout()
plt.show()

# --- 7E: Final summary figure ---
fig, axes = plt.subplots(2, 1, figsize=(13, 8), sharex=True)

# Panel 1: All three vol series — RV drawn last so it stays visible on top
axes[0].fill_between(dates,
                    sigma_hat - 1.96*np.sqrt(P_t),
                    sigma_hat + 1.96*np.sqrt(P_t),
                    alpha=0.12, color='navy', zorder=1)
axes[0].plot(dates, sigma_hat, lw=2.0, color='navy', label='Kalman  $\hat{\sigma}_{\{t\}}$ ', zorder=2)
axes[0].plot(dates, iv_t, lw=1.4, color='crimson', label='VIX (implied)', zorder=3)
axes[0].plot(dates, y_t, lw=0.9, color='steelblue', label='Realized Vol (raw)', alpha=0.75, zorder=4)

# Shade regimes
regime_bg = [('2020-01-01', '2021-01-01', 'lightyellow')]
for r_start, r_end, rc in regime_bg:
    axes[0].axvspan(pd.Timestamp(r_start), pd.Timestamp(r_end), color=rc, alpha=0.5, label='COVID crisis')

axes[0].set_ylabel('Volatility (decimal)')
axes[0].set_title('Latent Volatility as a Stochastic Process: Full Sample Results')
axes[0].legend(fontsize=9, loc='upper left')

# Panel 2: VRP signal
axes[1].fill_between(dates, vrp, 0,
                    where=(vrp>0), color='green', alpha=0.4, label='VRP > 0')
axes[1].fill_between(dates, vrp, 0,
                    where=(vrp<=0), color='red', alpha=0.3, label='VRP ≤ 0')
axes[1].plot(dates, vrp, lw=0.8, color='black', alpha=0.5)
axes[1].axhline(0, color='black', lw=0.8, ls='--')
axes[1].axhline(vrp.mean(), color='green', lw=1.2, ls=':',
                label=f'Mean VRP = {vrp.mean():.4f}')
axes[1].set_ylabel('VRP = VIX -  $\hat{\sigma}$ ')
axes[1].set_title(f'VRP Signal | Mean = {vrp.mean():.4f} | Sharpe = {sharpe:.3f} | NW t = {t_stat:.2f}')
axes[1].legend(fontsize=9, loc='upper left')
axes[1].xaxis.set_major_formatter(mdates.DateFormatter('%Y'))

plt.tight_layout()
plt.savefig('vrp_kalman_results.png', dpi=150, bbox_inches='tight')
plt.show()
print("\nFigure saved to vrp_kalman_results.png")
print("\nPipeline complete.")

# =====
# CELL 10 | Section 6.4 Extension — Kernel Baseline, Extended Regime Stats,
#         and March 2020 VRP Sign Flip Analysis
#
# Delivers three things the paper needs for the regime-centered narrative:
# (a) Gaussian kernel smoother as the static baseline (Section 6.4)
# (b) Per-regime VRP half-life and inter-regime Sharpe significance test
# (c) March 2020 sign-flip characterization for Section 6.3
# =====

from scipy.stats import ttest_ind

# -----
# 10A: Gaussian Kernel Smoother — Static Baseline (Section 6.4)
# -----

```

```

def gaussian_kernel_smooth(y, bandwidth=63):
    """
    Nadaraya-Watson kernel smoother with Gaussian kernel.
    Produces a static smoothed estimate of the vol series.
    Bandwidth h controls smoothness (in trading days, default ~1 quarter).
    """
    T = len(y)
    y_smooth = np.zeros(T)
    t_idx = np.arange(T, dtype=float)
    for t in range(T):
        weights = np.exp(-0.5 * ((t_idx - t) / bandwidth) ** 2)
        y_smooth[t] = np.sum(weights * y) / np.sum(weights)
    return y_smooth

print("Computing Gaussian kernel smoother (bandwidth = 63 days) ...")
sigma_kernel = gaussian_kernel_smooth(y_t, bandwidth=63)

vrp_kernel = iv_t - sigma_kernel
vrp_kalman = vrp # alias for clarity

sharpe_kernel = vrp_kernel.mean() / vrp_kernel.std() * np.sqrt(252)
sharpe_kalman_val = vrp_kalman.mean() / vrp_kalman.std() * np.sqrt(252)
sharpe_raw_val = vrp_raw.mean() / vrp_raw.std() * np.sqrt(252)

print("\n" + "=" * 65)
print(" Section 6.4 — Kalman vs Static Kernel vs Raw RV Baseline")
print("=" * 65)
print(f" {'Method':<30} {'Mean VRP':>10} {'Std VRP':>10} {'Sharpe':>8}")
print(" " + "-" * 62)
print(f" {'Kalman latent vol (dynamic)':<30} {vrp_kalman.mean():>10.5f} "
      f" {vrp_kalman.std():>10.5f} {sharpe_kalman_val:>8.3f}")
print(f" {'Kernel smoother (static)':<30} {vrp_kernel.mean():>10.5f} "
      f" {vrp_kernel.std():>10.5f} {sharpe_kernel:>8.3f}")
print(f" {'Raw rolling RV (naive)':<30} {vrp_raw.mean():>10.5f} "
      f" {vrp_raw.std():>10.5f} {sharpe_raw_val:>8.3f}")

print(f"\n Latent state noise reduction vs raw RV:")
print(f" Kalman sigma_hat std: {sigma_hat.std():.5f} "
      f"(reduction: {(1 - sigma_hat.std()/y_t.std())*100:.1f}%)")
print(f" Kernel sigma_hat std: {sigma_kernel.std():.5f} "
      f"(reduction: {(1 - sigma_kernel.std()/y_t.std())*100:.1f}%)")
print(f"\n Key distinction: the kernel smoother reduces noise by over-smoothing")
print(f" and loses responsiveness to structural breaks (e.g. COVID spike).")
print(f" The Kalman filter adapts dynamically via P_t, preserving")
print(f" responsiveness while filtering measurement noise.")

# -----
# 10B: Per-Regime Half-Life and Inter-Regime Sharpe Significance Test
# -----

print("\n" + "=" * 65)
print(" Extended Regime Analysis — Half-Life and Significance Tests")
print("=" * 65)

regime_vrp = {}
for label, mask in regimes.items():
    v = vrp_kalman[mask]
    rho = np.corrcorr(v[:-1], v[1:])[0, 1] if len(v) > 2 else np.nan
    hl = -np.log(2) / np.log(abs(rho)) if abs(rho) < 1 else np.inf
    sh = v.mean() / v.std() * np.sqrt(252)
    regime_vrp[label] = v

```



```

print(f"\n {label}")
print(f"   Mean: {v.mean():.4f} Std: {v.std():.4f} "
      f"Sharpe: {sh:.3f} Half-life: {hl:.1f} days AR(1) rho: {rho:.4f}")

# Pairwise Welch t-tests on regime VRP means
regime_labels = list(regime_vrp.keys())
print(f"\n Pairwise regime mean VRP difference tests (Welch t-test):")
print(f"   {'Comparison':<45} {'t-stat':>8} {'p-value':>10} {'Sig':>5}")
print(" " + "-" * 72)
pairs = [
    (regime_labels[0], regime_labels[1]),
    (regime_labels[0], regime_labels[2]),
    (regime_labels[1], regime_labels[2]),
]
for a, b in pairs:
    t_val, p_val_r = ttest_ind(regime_vrp[a], regime_vrp[b], equal_var=False)
    sig = '***' if p_val_r < 0.01 else '**' if p_val_r < 0.05 else '*' if p_val_r < 0.1 else "
    label_str = f"{a[:22]} vs {b[:22]}"
    print(f"   {label_str:<45} {t_val:>8.3f} {p_val_r:>10.4f} {sig:>5}")

# -----
# 10C: March 2020 VRP Sign Flip Analysis (Section 6.3)
# -----

print("\n" + "=" * 65)
print(" Section 6.3 — March 2020 VRP Sign Flip Characterization")
print("=" * 65)

crisis_mask = (dates >= '2020-02-01') & (dates <= '2020-05-31')
crisis_dates = dates[crisis_mask]
vrp_crisis = vrp_kalman[crisis_mask]
rv_crisis = y_t[crisis_mask]
vix_crisis = iv_t[crisis_mask]
sigma_crisis = sigma_hat[crisis_mask]

neg_mask = vrp_crisis < 0
neg_dates = crisis_dates[neg_mask]

print(f"\n Crisis window: Feb 1 to May 31 2020 ({crisis_mask.sum()} trading days)")
print(f" Days VRP < 0: {neg_mask.sum()} ({neg_mask.mean()*100:.1f}% of crisis window)")
if neg_mask.sum() > 0:
    print(f"   First negative: {neg_dates[0].date()}")
    print(f"   Last negative: {neg_dates[-1].date()}")
print(f"\n Peak realized vol: {rv_crisis.max():.4f} "
      f"({crisis_dates[rv_crisis.argmax()].date()})")
print(f" Peak VIX: {vix_crisis.max():.4f} "
      f"({crisis_dates[vix_crisis.argmax()].date()})")
print(f" Trough VRP: {vrp_crisis.min():.4f} "
      f"({crisis_dates[vrp_crisis.argmin()].date()})")
print(f" Peak VRP: {vrp_crisis.max():.4f} "
      f"({crisis_dates[vrp_crisis.argmax()].date()})")
print(f"\n Interpretation: realized vol peaked AFTER VIX, as the market")
print(f" priced in the crash before it fully materialized in returns.")
print(f" The Kalman filter latent state tracked the realized vol surge,")
print(f" temporarily compressing and inverting the VRP signal.")

# -----
# 10D: Plots
# -----

# Plot 1: Three vol estimators comparison
fig, axes = plt.subplots(2, 1, figsize=(13, 9), sharex=True)

```

```

axes[0].fill_between(dates,
                    sigma_hat - 1.96*np.sqrt(P_t),
                    sigma_hat + 1.96*np.sqrt(P_t),
                    alpha=0.10, color='navy', zorder=1)
axes[0].plot(dates, sigma_kernel, lw=1.6, color='darkorange',
            label='Kernel smoother (static)', ls='--', zorder=2)
axes[0].plot(dates, sigma_hat, lw=1.8, color='navy',
            label='Kalman sigma_hat (dynamic)', zorder=3)
axes[0].plot(dates, iv_t, lw=1.2, color='crimson',
            label='VIX (implied vol)', zorder=4)
axes[0].plot(dates, y_t, lw=0.8, color='steelblue', alpha=0.6,
            label='Realized Vol (raw)', zorder=5)
axes[0].axvspan(pd.Timestamp('2020-01-01'), pd.Timestamp('2021-01-01'),
            color='lightyellow', alpha=0.6, label='COVID crisis')
axes[0].set_ylabel('Volatility (decimal)')
axes[0].set_title('Section 6.4: Kalman Filter vs Kernel Smoother vs Raw RV')
axes[0].legend(fontsize=9, loc='upper left')

axes[1].plot(dates, vrp_kalman, lw=1.2, color='navy',
            label=f'Kalman VRP (Sharpe={sharpe_kalman_val:.2f})', zorder=3)
axes[1].plot(dates, vrp_kernel, lw=1.2, color='darkorange', ls='--',
            label=f'Kernel VRP (Sharpe={sharpe_kernel:.2f})', zorder=2, alpha=0.8)
axes[1].axhline(0, color='black', lw=0.8, ls='--')
axes[1].axhline(vrp_kalman.mean(), color='navy', lw=1.0, ls=':')
axes[1].axhline(vrp_kernel.mean(), color='darkorange', lw=1.0, ls=':')
axes[1].axvspan(pd.Timestamp('2020-01-01'), pd.Timestamp('2021-01-01'),
            color='lightyellow', alpha=0.6)
axes[1].set_ylabel('VRP = VIX - sigma_hat')
axes[1].set_title('VRP Signal: Dynamic (Kalman) vs Static (Kernel) Estimation')
axes[1].legend(fontsize=9)
axes[1].xaxis.set_major_formatter(mdates.DateFormatter('%Y'))

plt.tight_layout()
plt.savefig('vrp_kalman_vs_kernel.png', dpi=150, bbox_inches='tight')
plt.show()

# Plot 2: March 2020 closeup
fig, axes = plt.subplots(2, 1, figsize=(12, 7), sharex=True)

axes[0].plot(crisis_dates, rv_crisis, lw=1.2, color='steelblue',
            label='Realized Vol (raw)', alpha=0.8)
axes[0].plot(crisis_dates, sigma_crisis, lw=1.8, color='navy',
            label='Kalman sigma_hat')
axes[0].plot(crisis_dates, vix_crisis, lw=1.4, color='crimson',
            label='VIX (implied)')
axes[0].set_ylabel('Volatility (decimal)')
axes[0].set_title('Section 6.3: March 2020 VRP Sign Flip — Closeup')
axes[0].legend(fontsize=9)

axes[1].fill_between(crisis_dates, vrp_crisis, 0,
                    where=(vrp_crisis > 0), color='green', alpha=0.4, label='VRP > 0')
axes[1].fill_between(crisis_dates, vrp_crisis, 0,
                    where=(vrp_crisis <= 0), color='red', alpha=0.4, label='VRP ≤ 0')
axes[1].plot(crisis_dates, vrp_crisis, lw=1.2, color='black', alpha=0.7)
axes[1].axhline(0, color='black', lw=0.8, ls='--')
axes[1].set_ylabel('VRP = VIX - sigma_hat')
axes[1].set_title('VRP Sign Flip During COVID Crash')
axes[1].legend(fontsize=9)
axes[1].xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))

plt.tight_layout()

```

```
plt.savefig('vrp_march2020_closeup.png', dpi=150, bbox_inches='tight')
plt.show()

print("\nCell 10 complete.")
print("Figures saved: vrp_kalman_vs_kernel.png, vrp_march2020_closeup.png")
```